

EcoGotchi

Cuida. Juega. Ecoluciona.

EcoGotchi es tu mascota virtual que te acompaña para crear hábitos sostenibles.

Completa misiones, cuida del planeta y haz crecer un mundo mejor.



SOSTENIBILIDAD

Acciones que cuidan nuestro planeta



GAMIFICACIÓN

Misiones y retos divertidos



IMPACTO REAL

Pequeños hábitos, grandes cambios



HÁBITOS SALUDABLES

Mejora tu bienestar y el del planeta



¡Estoy genial!

MISIONES ECOLÓGICAS

Ahorrar agua

Reduce el tiempo en la ducha
+10 Monedas Verdes

Resolver misión

Reducir carne

Evita comer carne hoy para reducir huella ecológica
+15 Monedas Verdes

Resolver misión

Usar transporte publico

Evita el coche hoy
+20 Monedas Verdes

Resolver misión

ECO BARRA



INSTITUTO

IES Vista Alegre



CICLO FORMATIVO

FP DAW



PROYECTO

EcoGotchi

EQUIPO DE DESARROLLO



Cruz Robalino Amanda



García Seno,
Jose Manuel



Rodriguez Carreño,
Paloma

PEQUEÑOS HÁBITOS, GRANDES CAMBIOS.
TU MUNDO, TU IMPACTO.

INDICE

1. Introducción y contextualización

- 1.1. Descripción del problema
- 1.2. Motivación
- 1.3. Objetivos del proyecto
 - 1.3.1. Objetivo general
 - 1.3.2. Objetivos específicos
- 1.4. Estructura de la memoria

2. Análisis del sistema

- 2.1. Requisitos funcionales
- 2.2. Requisitos no funcionales
- 2.3. Casos de uso
 - Casos Clima: api.openweathermap.org
- 2.4. Planificación temporal

3. Diseño e Ingeniería de Software

- 3.1. Arquitectura general del sistema
 - URL de despliegue:
- 3.2. Evolución hacia patrón MVC
- 3.3. Estructura y organización del proyecto
 - 3.3.1. Organización general del proyecto
 - 3.3.2. Estructura MVC
 - 3.3.3. Recursos y configuración
 - 3.3.4. Base de Datos y Scripts SQL
 - 3.3.5. Archivos de entrada del sistema
 - 3.3.6. Organización y escalabilidad
- 3.4. Diseño de la base de datos
 - 3.4.1. Sistema gestor de base de datos
 - 3.4.2. Modelo relacional
- 3.5. Relación entre tablas principales
- 3.6. Flujo de funcionamiento del dashboard
- 3.7. Diagramas técnicos del sistema
 - 3.7.1. Diagrama simplificado arquitectura MVC
 - 3.7.2. Flujo Funcional el Dashboard
- 3.8. Decisiones de diseño y organización
 - 3.8.1. Decisión de utilizar arquitectura MVC
 - 3.8.2. Decisiones de diseño visual
 - 3.8.3. Decisiones sobre la jugabilidad
 - 3.8.4. Decisiones organizativas del equipo

4. Implementación técnica y Lógica de Negocio

- 4.1. Stack tecnológico avanzado
- 4.2. Arquitectura y estructura del proyecto (Patrón MVC)
- 4.3. Arquitectura Funcional y Módulos del Sistema

- 4.4. Base de Datos y Adaptación a Supabase
 - 4.4.1. Conexión Dual y Transaction Pooler
 - 4.4.2. Compatibilidad de Consultas SQL (MySQL vs PostgreSQL)
- 4.5. Lógica de Gamificación y Mecánicas Cliente-Servidor
 - 4.5.1. Degradación y Muerte de la Mascota
 - 4.5.2. Minijuegos y Entorno Interactivo
- 4.6. Control de Versiones y Gestión de Repositorios

5. Despliegue en Producción y Entorno Cloud

- 5.1. Vercel Serverless Runtime y Enrutamiento (Router API)
- 5.2. Optimización de Assets para Serverless
- 5.3. Integración Continua (CI/CD) y Variables de Entorno
- 5.4. Pruebas funcionales y de usuario.

6. Normativa, Ética y Licencias

- 6.1. Marco Legal: Privacidad y RGPD
- 6.2. Impacto Ético: Gamificación y ODS
- 6.3. Propiedad Intelectual

7. Conclusiones y Trabajo Futuro

- 7.1. Conclusiones y Cumplimiento de Objetivos
- 7.2. Lecciones Aprendidas
- 7.3. Hoja de ruta: Evolución y Escalabilidad
 - 7.3.1. Transición hacia Ecosistemas Móviles (PWA y Nativo)
 - 7.3.2. Convergencia con IoT (Internet de las Cosas)
 - 7.3.3. Dinámicas Sociales y Gamificación Grupal
 - 7.3.4. Inteligencia Artificial y Algoritmos Adaptativos
 - 7.3.5. Aseguramiento de la Calidad (QA) y Testing

8. Bibliografía y Referencias Consultadas

1. Introducción y contextualización

El siglo XXI se encuentra marcado por una crisis climática y ambiental sin precedentes. La degradación de los ecosistemas, el calentamiento global y la acumulación de residuos plásticos exigen una transformación profunda en los hábitos cotidianos de la ciudadanía. Sin embargo, a pesar de la amplia disponibilidad de información científica, la población suele enfrentarse a la apatía o a la dificultad de mantener una constancia en la adopción de prácticas sostenibles en su día a día. En este escenario nace **EcoGotchi**, una plataforma web interactiva diseñada para salvar la brecha entre la teoría ambiental y la acción práctica individual a través del juego.

1.1. Descripción del problema

La necesidad de concienciación ecológica es un hecho generalizadamente aceptado; no obstante, el verdadero reto reside en la modificación conductual a largo plazo. Las metodologías tradicionales de concienciación —tales como folletos informativos, charlas o documentales— poseen un impacto efímero. Los usuarios muestran una severa dificultad para mantener hábitos sostenibles debido a tres factores principales:

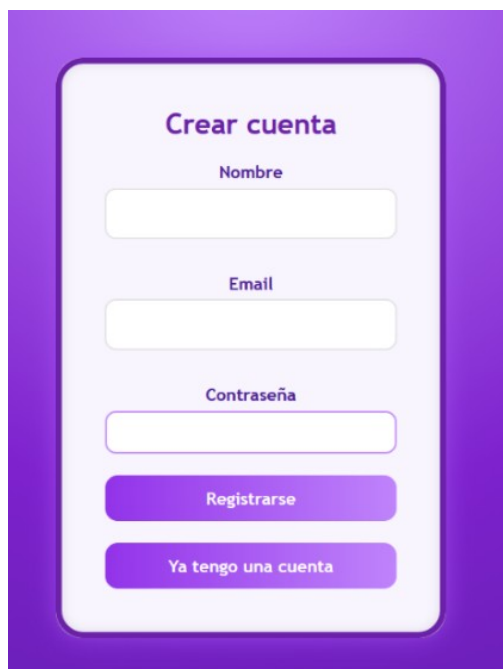
- **Ausencia de gratificación inmediata:** Las consecuencias de reciclar, ahorrar agua o reducir el consumo energético no son perceptibles de forma instantánea en el entorno del individuo, lo que debilita la motivación interna.
- **Falta de interactividad y seguimiento:** El ciudadano no dispone de herramientas accesibles que cuantifiquen el impacto positivo o negativo de sus rutinas diarias sobre un ecosistema vivo.
- **Complejidad percibida:** Muchas directrices medioambientales se perciben como obligaciones monótonas o restrictivas en lugar de integrarse orgánicamente en el estilo de vida digital actual.

1.2. Motivación

La motivación de este proyecto se fundamenta en la convergencia de la tecnología web moderna y la psicología conductual. Para transformar obligaciones ecológicas en acciones deseadas, se recurre a la **gamificación** (el uso de mecánicas de juego en entornos no lúdicos) y al concepto clásico de la **mascota virtual** como un recurso educativo disruptivo.

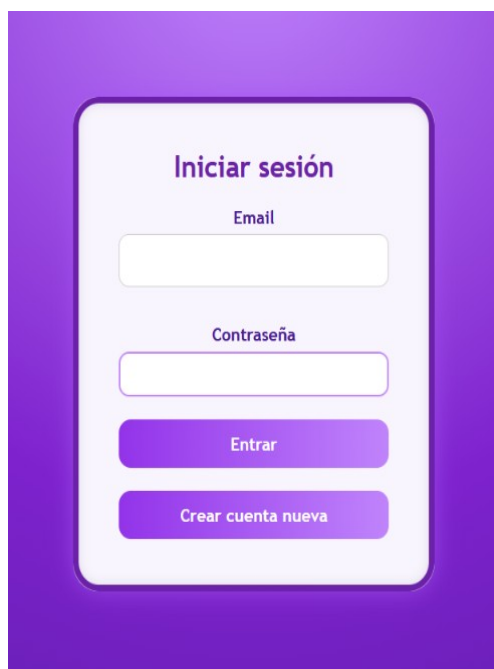
Al vincular el bienestar de una criatura digital al comportamiento ecológico real del usuario, se activa un mecanismo de *responsabilidad afectiva digital*. El jugador ya no

cuida un entorno abstracto, sino una mascota con nombre e identidad cuyo estado de salud depende de sus elecciones diarias. La gamificación proporciona un sistema inmediato de incentivos (puntos ecológicos y monedas virtuales) que sustituye la recompensa a largo plazo del planeta real por un refuerzo positivo directo en la pantalla del usuario, promoviendo un aprendizaje empírico y recurrente.



The illustration shows a mobile app interface for creating a new account. It features a purple gradient background. A white rounded rectangle contains the title 'Crear cuenta' in bold purple text. Below the title are three white input fields with purple borders, labeled 'Nombre', 'Email', and 'Contraseña' in small purple text. At the bottom of the form are two purple buttons with white text: 'Registrarse' and 'Ya tengo una cuenta'.

Ilustración 1. Crear Cuenta



The illustration shows a mobile app interface for logging in. It features a purple gradient background. A white rounded rectangle contains the title 'Iniciar sesión' in bold purple text. Below the title are two white input fields with purple borders, labeled 'Email' and 'Contraseña' in small purple text. At the bottom of the form are two purple buttons with white text: 'Entrar' and 'Crear cuenta nueva'.

Ilustración 2. Iniciar sesión



Ilustración 3. Adoptar mascota

Crea tu EcoMascota



Nombre de tu mascota

LUCY

Tipo de mascota

 Planta ▼

Color

Verde ▼

Verde

Morado

Azul

Ilustración 4. Crea tu EcoMascota 1



The form is titled "Crea tu EcoMascota" and features a small plant icon. It includes a text input field for the pet's name, which contains "LUCY". Below this is a dropdown menu for the pet type, currently set to "Planta". The dropdown list shows three options: "Planta", "Animal", and "Fantasía". At the bottom of the form is a purple button labeled "Crear Mascota".

Ilustración 5. Crea tu ecomascota 2

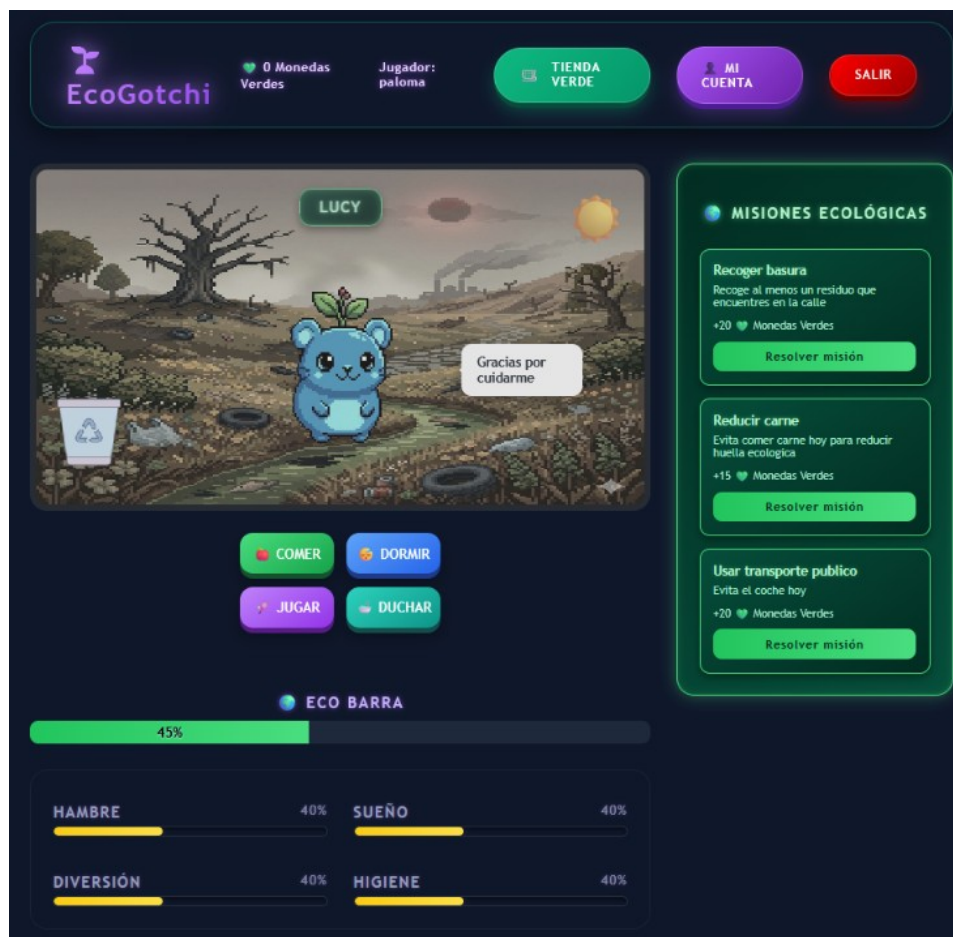


Ilustración 6. Dashboard Inicio

1.3. Objetivos del proyecto

1.3.1. Objetivo general

Desarrollar una aplicación web gamificada para fomentar hábitos sostenibles mediante el cuidado de una mascota virtual, implementada bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC), asegurando un entorno dinámico, interactivo, accesible y responsive en dispositivos móviles y de escritorio.

1.3.2. Objetivos específicos

- **Registro y autenticación de usuarios:** Diseñar un módulo seguro de control de acceso que permita la gestión individualizada del progreso de cada jugador mediante credenciales encriptadas.
- **Creación y cuidado de mascotas:** Desarrollar un sistema dinámico que permita adoptar una criatura virtual y gestionar sus parámetros vitales (hambre, sueño, diversión, higiene) sometidos a un algoritmo de degradación temporal en el servidor.
- **Sistema de misiones ecológicas:** Implementar un panel diario de retos sostenibles que el usuario debe validar para certificar su compromiso ambiental y eludir penalizaciones automáticas en el juego.
- **Tienda Verde con monedas:** Crear un entorno comercial cerrado dotado de una divisa interna ("monedas verdes"), obtenida mediante buenas prácticas, para la adquisición de ítems de cuidado y personalización.
- **Sistema de entorno ecológico dinámico:** Programar un motor visual que altere el fondo y la atmósfera del dashboard (bueno, normal, malo, extremo) en función del nivel ecológico acumulado.
- **Historial de mascotas:** Proveer un registro persistente que almacene la evolución y estado de las mascotas anteriores o del ciclo de vida del usuario en la base de datos.
- **Gestión de cuenta y baja de usuario:** Garantizar los derechos de privacidad y gestión de datos del usuario, habilitando un flujo de eliminación completa de su perfil y datos asociados.
- **Adaptación responsive:** Construir la capa de presentación utilizando técnicas avanzadas de maquetación CSS que garanticen una usabilidad óptima en teléfonos inteligentes, tabletas y ordenadores.

1.4. Estructura de la memoria

El presente documento técnico se articula de manera lógica en los siguientes bloques de contenido:

- **Capítulo 1: Introducción y contextualización:** Expone la problemática ambiental, la justificación lúdico-educativa del proyecto y el mapa de objetivos fijados.
- **Capítulo 2: Análisis del sistema:** Detalla la especificación de requisitos funcionales y no funcionales, el modelado formal mediante casos de uso y la cronología real de ejecución.
- **Capítulo 3: Diseño e Ingeniería de Software:** Describe la arquitectura cliente-servidor global, los diagramas lógicos y la estructuración del patrón MVC adoptado de manera unificada por el equipo.
- **Capítulo 4: Implementación técnica y Lógica de Negocio:** Desglosa el desarrollo de código fuente backend (PHP) y frontend (JS Vanilla), algoritmos de tiempo y la persistencia de datos.
- **Capítulo 5: Despliegue en producción:** Detalla los pasos para la publicación en producción cloud y configuraciones operativas.
- **Capítulo 6: Normativa, Ética y Licencias:** Marco legal, impacto ético y propiedad intelectual.
- **Capítulo 7: Conclusiones y líneas futuras:** Ofrece una valoración crítica del desarrollo y propone extensiones del ecosistema EcoGotchi.

2. Análisis del sistema

Este capítulo delimita el alcance analítico del proyecto web, abstrayendo las necesidades operativas de EcoGotchi en un conjunto rígido de directrices técnicas que guiarán la programación del sistema.

2.1. Requisitos funcionales

Los requisitos funcionales determinan los servicios y operaciones explícitas que provee la aplicación:

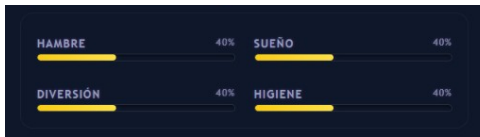
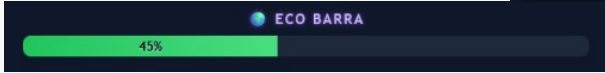
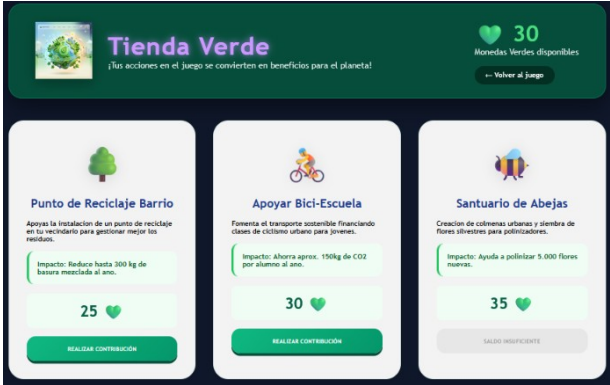
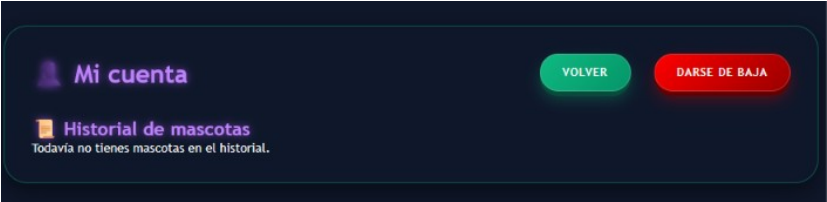
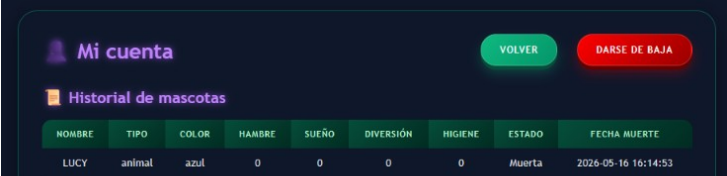
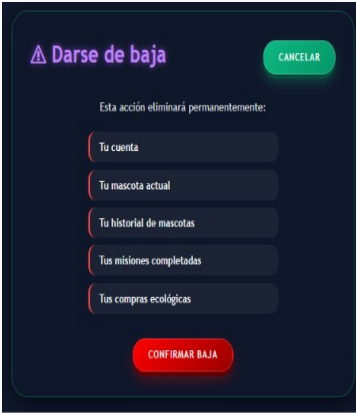
Código	Requisito Funcional	Descripción Técnica Detallada
RF-01	Registro e inicio de sesión	Permite la creación de cuentas de usuario únicas mediante credenciales seguras y el control de accesos simultáneos a la plataforma.
RF-02	Adopción de EcoMascota	El usuario debe seleccionar una especie inicial (animal o planta) y asignarle un nombre al iniciar su experiencia por primera vez.
RF-03	Cuidar mascota	El usuario aplicará consumibles para restaurar el Hambre, Sueño, Diversión e Higiene. Las peticiones se ejecutarán en segundo plano asincrónicamente mediante Fetch API.  Ilustración SEQ Ilustración * ARABIC 7. Barras
RF-04	Completar misiones Ecológicas	Un panel diario listará las metas ambientales. Al marcarlas como completadas, el sistema incrementará el nivel ecológico global y asignará monedas verdes en la cuenta del jugador.  Ilustración SEQ Ilustración * ARABIC 8. Ejemplo de misión

		Ilustración SEQ Ilustración * ARABIC 9. Misiones ecologicas Ilustración SEQ Ilustración * ARABIC 11. Monedas en Tienda  Ilustración SEQ Ilustración * ARABIC 10. Ecobarra
RF-05	Comprar acciones en Tienda	Interfaz de intercambio comercial donde las monedas acumuladas se canjean por consumibles (comida, medicinas, elementos estéticos)..  Ilustración SEQ Ilustración * ARABIC 12. Tienda verde
RF-06	Ver dashboard	Panel consolidado donde convergen las barras de vida de la mascota, el widget de clima dinámico por API externa, la tienda emergente y el inventario en una única vista.
RF-07	Ver historial de mascotas	Registro y persistencia de las mascotas previas que han completado su ciclo vital o han fallecido por negligencia en el cuidado del usuario, almacenados históricamente en el servidor.  Ilustración SEQ Ilustración * ARABIC 13. Historial de mascotas 1  Ilustración SEQ Ilustración * ARABIC 14. Historial de mascotas 2

RF-08	Darse de baja	Acción de borrado destructivo (DELETE cascade) que elimine permanentemente al usuario, sus mascotas activas y sus transacciones de la base de datos relacional. Ilustración SEQ Ilustración * ARABIC 15. Darse de Baja	
RF-09	Consultar estado ecológico	Lectura constante del indicador ecológico para condicionar las reglas de negocio en PHP y mutar las texturas de entorno mostradas al usuario en el frontend.	

2.2. Requisitos no funcionales

Los requisitos no funcionales determinan restricciones cualitativas del sistema web:

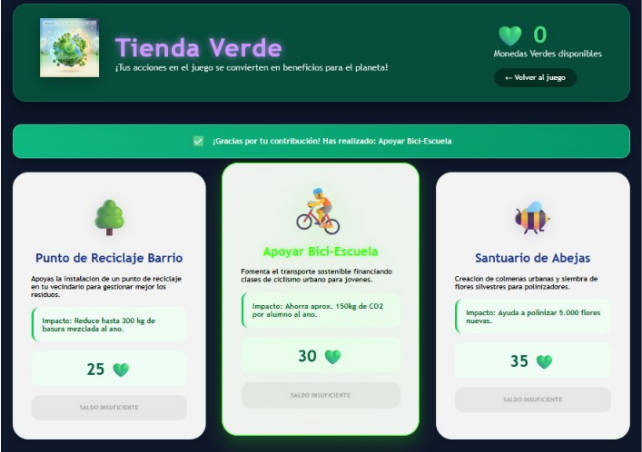
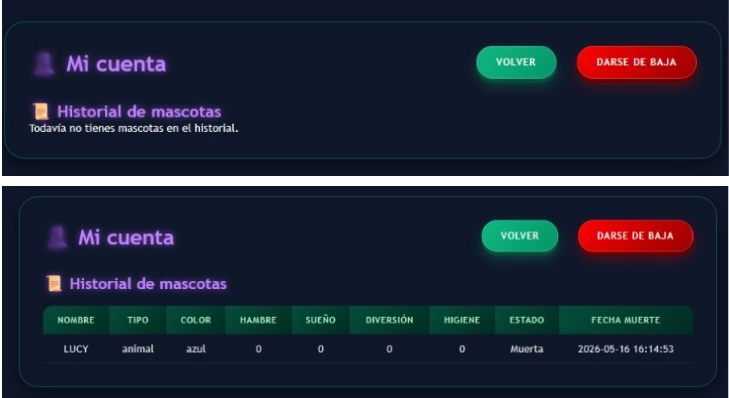
- **Usabilidad:** Curva de aprendizaje intuitiva, botones interactivos con retroalimentación inmediata (hover, deshabilitación de botones tras pulsación) y un diseño cromático basado en estándares web para mitigar la fatiga visual.
- **Seguridad básica:** Protección frente a inyecciones SQL usando sentencias preparadas nativas mediante PDO. Sanitización de entradas del usuario para evitar ataques XSS y almacenamiento seguro de datos de sesión.
- **Persistencia de datos:** Base de datos relacional MySQL/MariaDB parametrizada para garantizar la integridad referencial y mantener los estados vitales a salvo de cierres inesperados de navegador.
- **Diseño responsive:** Estilos adaptativos definidos mediante Flexbox, Grid y Media Queries que aseguren una experiencia fluida e idéntica en pantallas táctiles verticales y monitores apaisados.
- **Mantenibilidad mediante MVC:** Estricta separación del software. La lógica de persistencia reside en Modelos, las interfaces en Vistas y el flujo intermedio en Controladores independientes, facilitando refactorizaciones locales sin afectar al ecosistema global.
- **Disponibilidad mediante despliegue en Vercel:** El aplicativo web debe desplegarse en una infraestructura cloud escalable que asegure tiempos de operatividad elevados y enrutamiento ágil a través de ficheros de configuración específicos del proveedor.

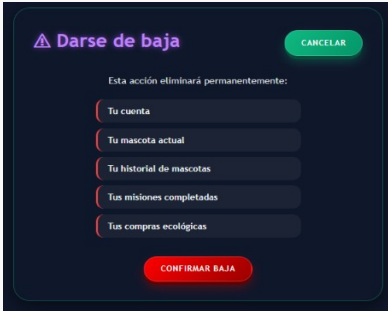
2.3. Casos de uso

Para detallar el comportamiento del sistema ante las interacciones de los actores, se definen los siguientes casos de uso principales ejecutados por el **Usuario Registrado**:

Caso de Uso	Flujo Principal de Acciones	Excepciones y Validación Técnica
CU-01: Registrar usuario	1. El visitante introduce email, nombre y password. 2. Envía formulario. 3. El sistema crea la cuenta.	Si el email ya existe, el controlador interrumpe el proceso y retorna un mensaje de error a la vista.
CU-02: Iniciar sesión	1. Introduce credenciales. 2. El sistema valida mediante password_verify(). 3. Redirección al Dashboard.	Si las credenciales son erróneas, se deniega el acceso sin desvelar qué campo falló por seguridad.
CU-03: Adoptar mascota	1. El usuario introduce el nombre de la mascota. 2. Escoge el tipo/color. 3. Envía los datos.	Si el nombre está vacío o infringe la longitud permitida, el modelo rechaza la inserción SQL.
CU-04: Cuidar mascota	1. Pulsa un ítem del inventario (e.g., alimentar). 2. JS envía petición asíncrona. 3. Las barras vitales suben.	Si el ítem se agota en el inventario, el controlador bloquea la acción devolviendo un código HTTP 400.
CU-05: Completar misión	1. Marca casilla de acción ecológica. 2. Servidor procesa	No se permite re-completar la misma misión dentro del mismo ciclo diario de 24 horas.

	la bonificación. 3. Sube el Nivel Eco y Monedas.	
--	--	--

Caso de Uso	Flujo Principal de Acciones	Excepciones y Validación Técnica
CU-06: Comprar acción	1. Clica en comprar ítem en la tienda. 2. El sistema verifica fondos. 3. Transfiere el ítem al inventario.	Si las Monedas Verdes son insuficientes, se cancela la transacción y se notifica con un mensaje visual.  Ilustración SEQ Ilustración * ARABIC 16. Comprar Acción
CU-07: Consultar historial	1. Accede a la pestaña histórica. 2. El sistema lee registros antiguos de la BBDD. 3. Renderiza la lista.	4. Si es un usuario nuevo sin registros previos, se muestra un mensaje informativo de "Historial vacío". Ilustración SEQ Ilustración * ARABIC 17. Consultar historial 

CU-08: Darse baja	de 1. Clica en "Eliminar cuenta". 2. Confirma la acción desde la pantalla de baja de usuario. 3. El sistema purga las tablas.	La acción es irreversible y destruye la sesión del navegador de manera inmediata.  Ilustración SEQ Ilustración * ARABIC 18. Darse de Baja
CU-09: Cerrar sesión	1. Pulsa "Salir". 2. El servidor destruye la sesión activa. 3. Redirección automática a login.php.	Intentar regresar mediante las flechas del navegador forzará la redirección al login al carecer de token.  Ilustración SEQ Ilustración * ARABIC 19. Login

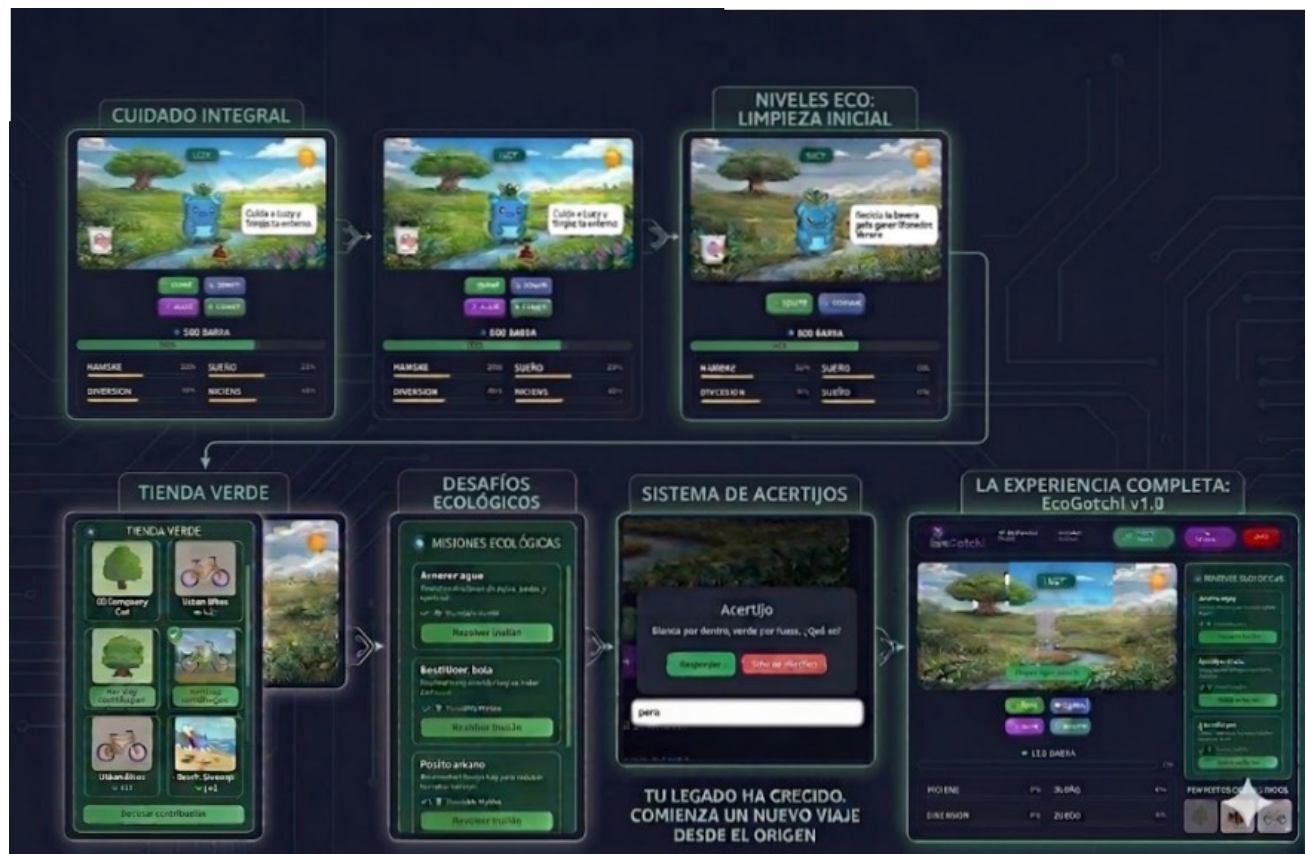


Ilustración 20. Evolucion APP

Casos Clima: api.openweathermap.org

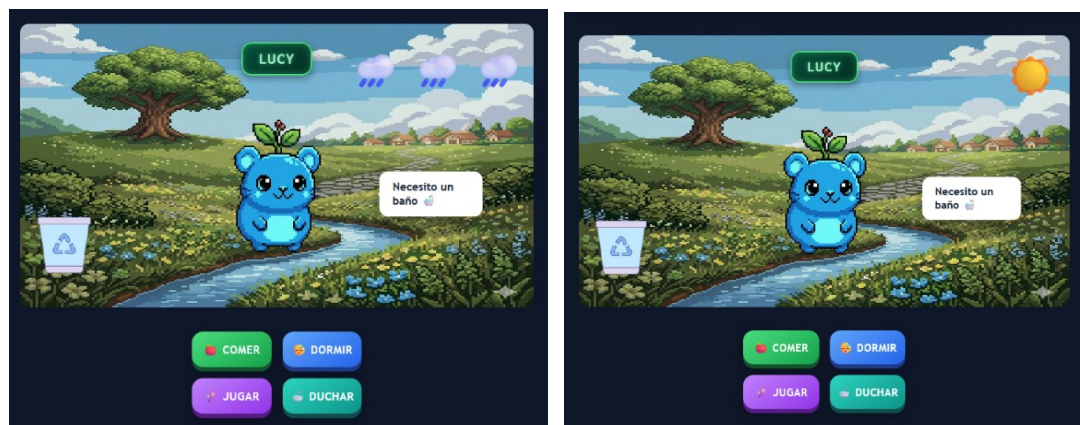


Ilustración 21. LLuvia

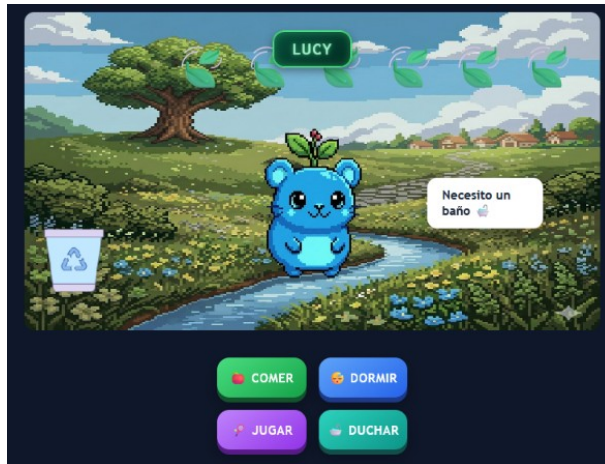


Ilustración 22. Sol

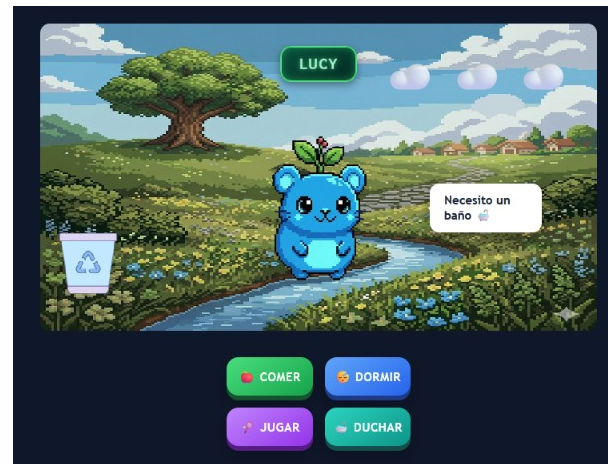


Ilustración 23. Viento

Ilustración 24. Nubes

2.4. Planificación temporal

El ciclo de desarrollo de EcoGotchi se organizó de forma iterativa e incremental a lo largo de las siguientes fases reales de trabajo estructurado:

1. **Fase de Diseño Inicial:** Conceptualización de la idea de negocio ambiental, bocetado de las interfaces en papel/herramientas de prototipado, estructuración lógica del esquema entidad-relación de la base de datos y definición preliminar del alcance técnico.
2. **Fase de Desarrollo Funcional:** Programación procedural del núcleo del simulador. Establecimiento de los scripts de conexión, algoritmos de degradación temporal de estadísticas vitales (hambre, sueño, diversión, higiene) basados en marcas de tiempo reales en el servidor.
3. **Fase de Tienda y Misiones:** Construcción e integración de las mecánicas principales de gamificación. Desarrollo de la lógica de transacciones comerciales con monedas verdes, el inventario de consumibles y la validación de retos sostenibles diarios con persistencia en MySQL.
4. **Fase de Refactorización a MVC:** Reestructuración integral de la base de código existente para adaptarla al patrón de diseño unificado del equipo (Modelo-Vista-Controlador). Separación estricta de las consultas SQL, la maquetación estética de las vistas y el control del flujo lógico mediante enrutadores en PHP.
5. **Fase de Mejoras Finales:** Pulido de la experiencia de usuario frontend mediante JavaScript Vanilla (Fetch API asíncrono para evitar recargas),

integración con la API externa de meteorología e implementación del diseño adaptativo responsive.

6. **Fase de Despliegue:** Migración y configuración del proyecto desde el entorno local (XAMPP) hacia infraestructuras en producción cloud utilizando Vercel, optimizando las variables de entorno de producción y la compatibilidad del enrutamiento de URLs.
7. **Fase de Memoria:** Redacción técnica de la documentación, consolidación de la arquitectura de software, especificación de requisitos y ensamblado del documento final del proyecto.

3. Diseño e Ingeniería de Software

3.1. Arquitectura general del sistema

EcoGotchi se ha desarrollado como una aplicación web basada en una arquitectura cliente-servidor. Este modelo permite separar el procesamiento de la lógica del negocio del entorno visual mostrado al usuario.

En el lado del cliente (front-end), la interfaz ha sido desarrollada utilizando HTML, CSS y JavaScript. HTML se encarga de estructurar el contenido de las páginas, CSS proporciona el diseño visual y las animaciones del entorno, mientras que JavaScript aporta interactividad en determinadas acciones del juego.

En el lado del servidor (back-end), se ha utilizado PHP como lenguaje principal de programación. PHP se encarga de procesar las peticiones del usuario, gestionar las sesiones, controlar la lógica de las mascotas y comunicarse con la base de datos.

Para el almacenamiento persistente de la información se ha utilizado MySQL/MariaDB, permitiendo guardar usuarios, mascotas, misiones, compras y el progreso ecológico de cada partida.

Durante el desarrollo del proyecto, la estructura inicial basada en archivos PHP más procedurales evolucionó hacia una arquitectura organizada mediante el patrón MVC (Modelo-Vista-Controlador). Esta reorganización permitió mejorar la mantenibilidad del código, separar responsabilidades y facilitar futuras ampliaciones del sistema.

La aplicación se ejecutó localmente mediante XAMPP utilizando Apache y MySQL. Posteriormente, el proyecto fue desplegado en producción mediante la plataforma Vercel, permitiendo su acceso público desde cualquier navegador web a través de Internet.

URL de despliegue:

<https://ecogotchi.vercel.app>

3.2. Evolución hacia patrón MVC

A medida que el proyecto fue creciendo y aumentando el número de funcionalidades implementadas, surgió la necesidad de reorganizar el código para mejorar su mantenimiento, comprensión y escalabilidad. Por este motivo, el equipo decidió adaptar progresivamente la aplicación hacia una arquitectura inspirada en el patrón MVC (Modelo - Vista - Controlador).

La aplicación comenzó inicialmente con una estructura más sencilla basada en archivos PHP independientes. Sin embargo, conforme se añadieron nuevas funcionalidades

relacionadas con la mascota virtual, las misiones ecológicas, la Tienda Verde y el sistema de usuarios, se hizo necesario separar mejor las responsabilidades del sistema.

La arquitectura implementada no utiliza frameworks completos como Laravel o Symfony, sino una adaptación manual de MVC utilizando PHP estructurado.

El sistema quedó dividido en tres capas principales:

- Los controladores (app/controllers) gestionan las acciones del usuario y coordinan la comunicación entre modelos y vistas.
- Los modelos (app/models) contienen la lógica del negocio y el acceso a la base de datos.
- Las vistas (app/views) se encargan de mostrar la interfaz visual utilizando HTML, CSS y fragmentos dinámicos en PHP.

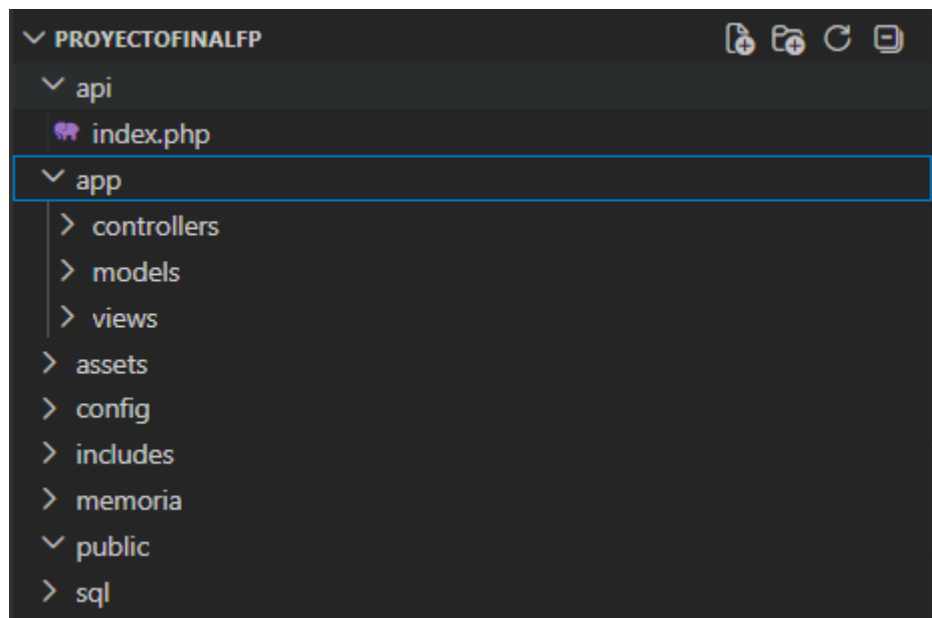


Ilustración 25. Estructura MVC

Esta reorganización permitió mejorar considerablemente la estructura interna del proyecto y facilitar el desarrollo colaborativo entre los miembros del equipo. Además, ayudó a reutilizar lógica común y mantener el código más limpio y modular.

Durante esta transición, algunos archivos procedurales originales continuaron utilizándose como puntos de entrada ligeros hacia los controladores. Estos archivos reciben las peticiones del navegador y posteriormente delegan la lógica principal en las clases correspondientes del sistema.

Aunque una arquitectura MVC completa suele centralizar todas las peticiones en un único index.php, en este proyecto se optó por una solución intermedia más sencilla y adaptada al

alcance académico del trabajo, manteniendo compatibilidad con parte de la estructura desarrollada inicialmente.

La adopción de esta arquitectura permitió separar adecuadamente las distintas responsabilidades del sistema y sentó una base más organizada para futuras ampliaciones de la aplicación.

3.3. Estructura y organización del proyecto

Con el objetivo de mantener una aplicación organizada y facilitar el mantenimiento del código, el proyecto fue estructurado utilizando una distribución modular de carpetas y archivos. Esta organización permitió separar las distintas responsabilidades del sistema y mejorar el trabajo colaborativo entre los miembros del equipo.

La estructura general del proyecto se divide en varios bloques funcionales principales.

3.3.1. Organización general del proyecto

El proyecto se organiza principalmente alrededor de varias carpetas que separan la lógica del negocio, la interfaz visual, la configuración del sistema y los recursos estáticos utilizados por la aplicación.

Entre las carpetas más importantes destacan:

- /app
- /assets
- /config
- /includes
- /sql
- /api

Esta estructura permitió mantener una distribución más clara del código y evitar que toda la lógica quedase concentrada en archivos PHP independientes como ocurría en las primeras fases del desarrollo.

3.3.2. Estructura MVC

La carpeta /app contiene la organización principal inspirada en el patrón MVC.

Controladores (/app/controllers)

Los controladores gestionan las acciones realizadas por el usuario y coordinan la comunicación entre vistas y los modelos del sistema.

Entre los controladores principales utilizados en la aplicación se encuentran:

-AuthController

- DashboardController
- MascotaController
- MisionController
- TiendaController
- UsuarioController
- HistorialController

Estos controladores permiten centralizar funcionalidades relacionadas con el login, el dashboard principal, las acciones de la mascota, las misiones ecológicas, la tienda virtual y la gestión del historial de usuarios.

Modelos (/app/models)

Los modelos contienen la lógica del negocio y las operaciones relacionadas con la base de datos.

Dentro de esta carpeta se encuentran clases como:

- Usuario
- 1. Mascota
- 2. Entorno
- 3. Mision
- 4. MisionCompletada
- 5. Tienda
- 6. Clima

Estas clases gestionan funcionalidades como:

1. autenticación de usuarios,
2. degradación automática de estadísticas,
3. control del entorno ecológico,
4. obtención de misiones,
5. historial de mascotas,
6. compras de la tienda verde,
7. conexión con APIs externas relacionadas con el clima

Vistas (/app/views)

Las vistas contienen la interfaz visual mostrada al usuario final.

Se encuentran organizadas por módulos funcionales como:

- auth
- mascota
- usuario
- tienda

Dentro de estas vistas se utilizan HTML, CSS y fragmentos dinámicos de PHP para representar la información generada por los controladores y modelos.

3.3.3. Recursos y configuración

Carpeta /assets

La carpeta /assets almacena los recursos gráficos y archivos estáticos utilizados por la aplicación.

Dentro de ella destacan:

- imágenes de mascotas,
- fondos ecológicos,
- elementos visuales de la tienda,
- iconos,
- hojas de estilos CSS,
- y scripts javascript complementarios.

El archivo principal de estilos (styles.css) concentra gran parte del diseño visual del proyecto, incluyendo:

- dashboard principal,
- sistema responsive,
- barras de estadísticas,
- animaciones,
- clima dinámico,
- tienda,
- historial,
- y gestión de usuario.

Carpeta /config

La carpeta /config contiene la configuración principal del sistema.

El archivo más importante es:

- database.php

Este archivo centraliza la conexión con la base de datos MySQL y permite reutilizar la conexión desde distintos módulos del sistema.

Carpeta /includes

La carpeta /includes contiene archivos auxiliares reutilizables utilizados por diferentes partes de la aplicación.

Entre ellos destaca:

- proteger.php

Este archivo se utiliza para verificar que el usuario haya iniciado sesión antes de acceder a determinadas páginas privadas del sistema.

3.3.4. Base de Datos y Scripts SQL

La carpeta /sql contiene los scripts utilizados para crear y configurar la base de datos del proyecto.

Entre los archivos incluidos destacan:

- ecogotchi.sql
- ecogotchi_supabase.sql

Estos scripts permiten generar las tablas necesarias para el funcionamiento de la aplicación y facilitan la migración o recreación del entorno de desarrollo.

La base de datos almacena información relacionada con:

- usuarios,
- mascotas,
- historial,
- misiones,
- entorno ecológico,
- compras,
- progreso del juego.

3.3.5. Archivos de entrada del sistema

A pesar de la reorganización MVC, el proyecto mantiene algunos archivos PHP procedurales utilizados como puntos de entrada ligeros para determinadas acciones del sistema.

Entre ellos se encuentran:

- login.php
- dashboard.php
- accion_mascota.php
- comprar_accion.php
- completar_mision.php
- baja_usuario.php

Estos archivos reciben las peticiones del navegador e instancian posteriormente los controladores correspondientes, delegando en ellos la lógica principal de la aplicación.

Esta solución permitió mantener compatibilidad con parte de la estructura original del proyecto mientras se realizaba la transición progresiva hacia una organización más modular basada en MVC.

3.3.6. Organización y escalabilidad

La organización modular del proyecto permitió mejorar considerablemente la mantenibilidad del código y facilitó la incorporación de nuevas funcionalidades durante el desarrollo.

Además, esta estructura favoreció el trabajo colaborativo entre los miembros del equipo, permitiendo que distintas partes del sistema pudieran desarrollarse de forma más independiente y ordenada.

3.4. Diseño de la base de datos

3.4.1. Sistema gestor de base de datos

El proyecto utiliza un sistema gestor de bases de datos relacional MySQL/MariaDB para almacenar de forma persistente toda la información necesaria para el funcionamiento de la aplicación.

La base de datos permite gestionar el estado de las mascotas virtuales, las cuentas de usuario, las misiones ecológicas, las compras realizadas en la Tienda Verde y el historial completo de cada partida.

Durante el desarrollo se trabajó principalmente utilizando MariaDB integrado en XAMPP, facilitando el entorno local de desarrollo junto con Apache y PHP.

El uso de una base de datos relacional permitió organizar correctamente las distintas entidades del sistema y establecer relaciones entre usuarios, mascotas, misiones y acciones realizadas dentro de la plataforma.

Además, la persistencia de datos resultó fundamental para conservar el progreso de cada usuario entre sesiones, permitiendo mantener estadísticas, monedas verdes, historial de mascotas y evolución del entorno ecológico incluso después de cerrar la aplicación.

3.4.2. Modelo relacional

La estructura de la base de datos se diseñó siguiendo un modelo relacional, permitiendo organizar la información en diferentes tablas conectadas entre sí mediante claves primarias y claves foráneas.

Este enfoque facilitó mantener la integridad de los datos y permitió relacionar correctamente los distintos elementos del sistema, como usuarios, mascotas, misiones, compras y evolución del entorno ecológico.

El modelo relacional facilitó separar adecuadamente las distintas entidades de la aplicación y evitar duplicidad innecesaria de información, mejorando además la escalabilidad y organización general del sistema.

La persistencia de datos resulta fundamental dentro de EcoGotchi, ya que permite conservar el progreso de cada usuario entre sesiones, incluyendo estadísticas de la mascota, monedas verdes, historial y evolución del entorno ecológico.

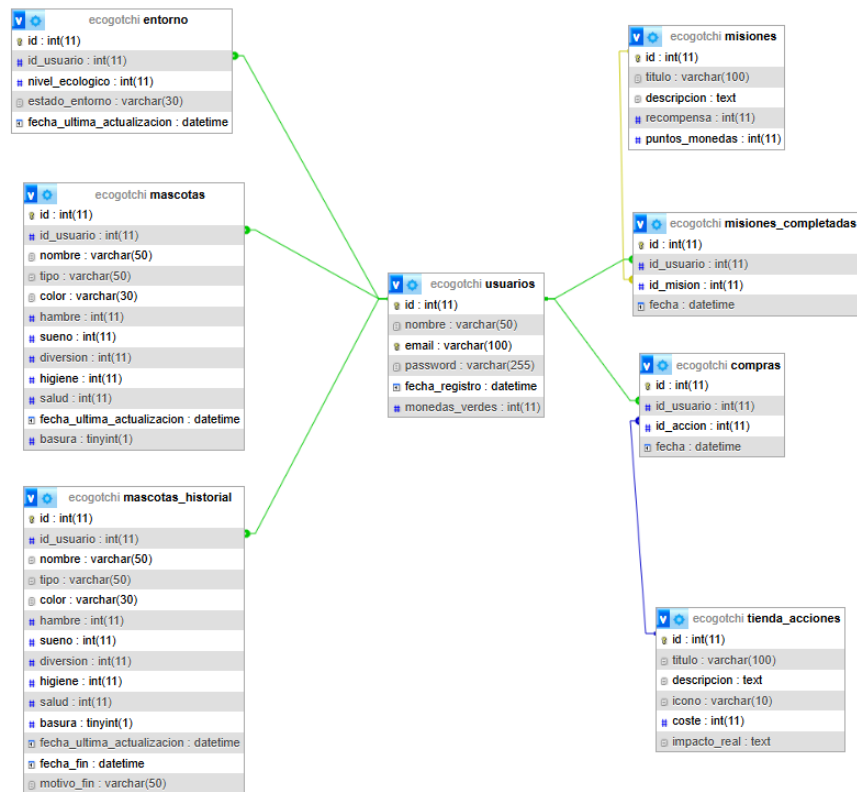


Ilustración 26. Modelo relacional-Relación entre tablas

3.5. Relación entre tablas principales

La entidad principal del sistema es la tabla usuarios, ya que cada usuario registrado dispone de información asociada con su progreso dentro de la aplicación.

A partir de esta tabla se establecen distintas relaciones con otras entidades del sistema:

- mascotas: almacena la mascota activa del usuario y sus estadísticas principales.
- entorno: guarda el nivel ecológico y el estado ambiental asociado a cada usuario.
- misiones_completadas: registra las misiones ecológicas realizadas por cada jugador.
- compras: almacena las acciones y mejoras adquiridas en la Tienda Verde.
- mascotas_historial: conserva información histórica de mascotas anteriores y partidas finalizadas.

Además, el sistema incluye tablas auxiliares independientes como:

- misiones: contiene las misiones ecológicas disponibles dentro del juego.
- tienda_acciones: almacena las acciones y objetos disponibles en la Tienda Verde.

Las relaciones implementadas permiten que un mismo usuario pueda mantener su progreso completo dentro de la plataforma, registrando tanto la evolución de su mascota como las acciones ecológicas realizadas durante el juego.

El modelo relacional también facilitó implementar funcionalidades como:

- historial de mascotas fallecidas,
- sistema de monedas verdes,
- registro de compras,
- control de degradación de estadísticas,
- persistencia del entorno ecológico del usuario.

3.6. Flujo de funcionamiento del dashboard

El dashboard es la pantalla principal de EcoGotchi y actúa como centro de control del juego. Desde esta vista el usuario puede consultar el estado de su mascota, ver el nivel ecológico, completar misiones, acceder a la Tienda Verde y gestionar su cuenta.

El flujo comienza en el archivo dashboard.php, que funciona como punto de entrada. Este archivo incluye la protección de sesión mediante includes/proteger.php y posteriormente instancia el DashboardController, delegando en él la lógica principal de la pantalla.

Una vez cargado el controlador, el sistema obtiene el identificador del usuario autenticado mediante la variable de sesión `$_SESSION["usuario_id"]`. A partir de este identificador se consultan los datos necesarios para construir el dashboard.

En primer lugar, el controlador obtiene la mascota activa del usuario mediante el modelo Mascota. También obtiene el entorno ecológico asociado al usuario mediante el modelo Entorno. Si el usuario todavía no dispone de entorno, este se crea automáticamente con un ecológico inicial de 50 y estado normal.

Después, si existe una mascota activa, el sistema calcula cuanto tiempo ha pasado desde la última actualización registrada. Para ello se utiliza la fecha almacenada en la base de datos y se calcula la diferencia en minutos. Si ha transcurrido tiempo, se aplica la degradación automática de estadísticas:

- el hambre baja 2 puntos por minuto,

- el sueño baja 1 punto por minuto,
- la diversión baja 2 puntos por minuto,
- la higiene baja 1 punto por minuto.

Estas estadísticas se actualizan en la base de datos antes de mostrarse al usuario. De esa forma, la mascota evoluciona aunque el usuario no haya estado conectado.

A continuación, el sistema comprueba si la mascota ha muerto. En la versión actual la mascota se considera muerta cuando hambre, sueño, diversión e higiene llegan a 0. Cuando esto ocurre, la mascota se guarda previamente en la tabla de historial, se elimina de la tabla de mascotas activas y se reinicia el entorno ecológico del usuario.

Después se gestiona el sistema de misiones. El controlador consulta cuántas misiones ha completado el usuario durante el día. Si no ha completado ninguna, se aplica una penalización del día para mostrarlas en el panel lateral del dashboard.

El controlador también prepara las variables visuales que utilizará la vista. Para ello, el modelo Entorno calcula el estado visual del entorno y la imagen de fondo correspondiente según el nivel ecológico. También se calcula el color de la barra ecológica de forma dinámica.

Además, el modelo Mascota genera el mensaje que mostrará la mascota y construye el nombre de la imagen correspondiente a partir del tipo y color de la mascota. Por otro lado, el modelo Clima consulta la API meteorológica y devuelve una clase CSS que permite mostrar efectos visuales de clima, como sol, lluvia y nubes.

Finalmente, el controlador obtiene las monedas verdes del usuario y carga la vista `app/views/mascota/dashboard.php`. Esta vista recibe todas las variables preparadas por el controlador y se encarga de mostrar la interfaz final: mascota, barras de estadísticas, nivel ecológico, misiones, clima, tienda, historial, y opciones de usuario.

Este flujo permite que el dashboard funcione como una pantalla dinámica conectada con la base de datos, manteniendo separada la lógica principal en los modelos y controladores, y dejando a la vista la responsabilidad de representar la información al usuario

3.7. Diagramas técnicos del sistema

Con el objetivo de facilitar la comprensión técnica de la aplicación, se elaboraron distintos diagramas que representan la arquitectura MVC implementada en el sistema y el flujo principal de funcionamiento del dashboard.

Estos diagramas permiten visualizar de forma más clara la organización interna del proyecto, la comunicación entre controladores, modelos y vistas, así como el procesamiento general de las principales funcionalidades de EcoGotchi.

3.7.1. Diagrama simplificado arquitectura MVC

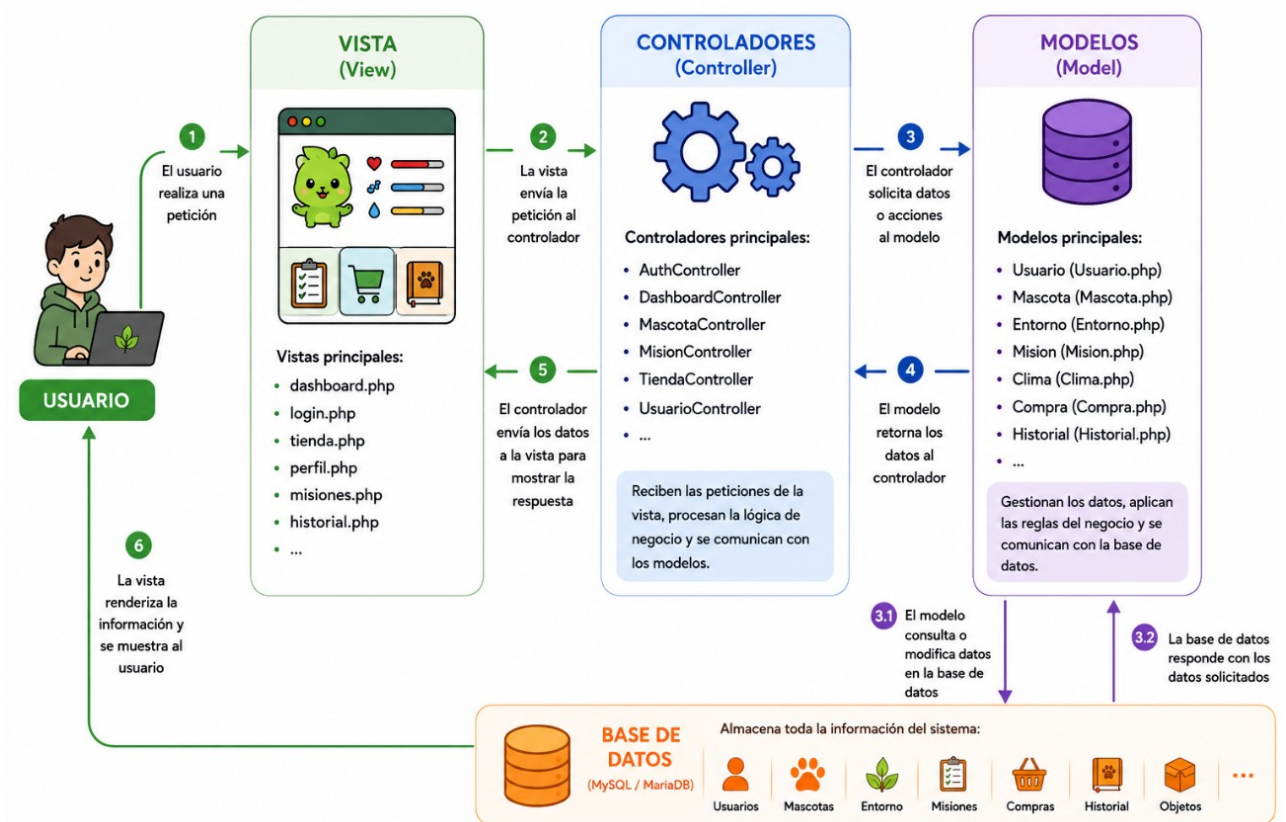


Ilustración 27. Diagrama simplificado arquitectura MVC

3.7.2. Flujo Funcional el Dashboard

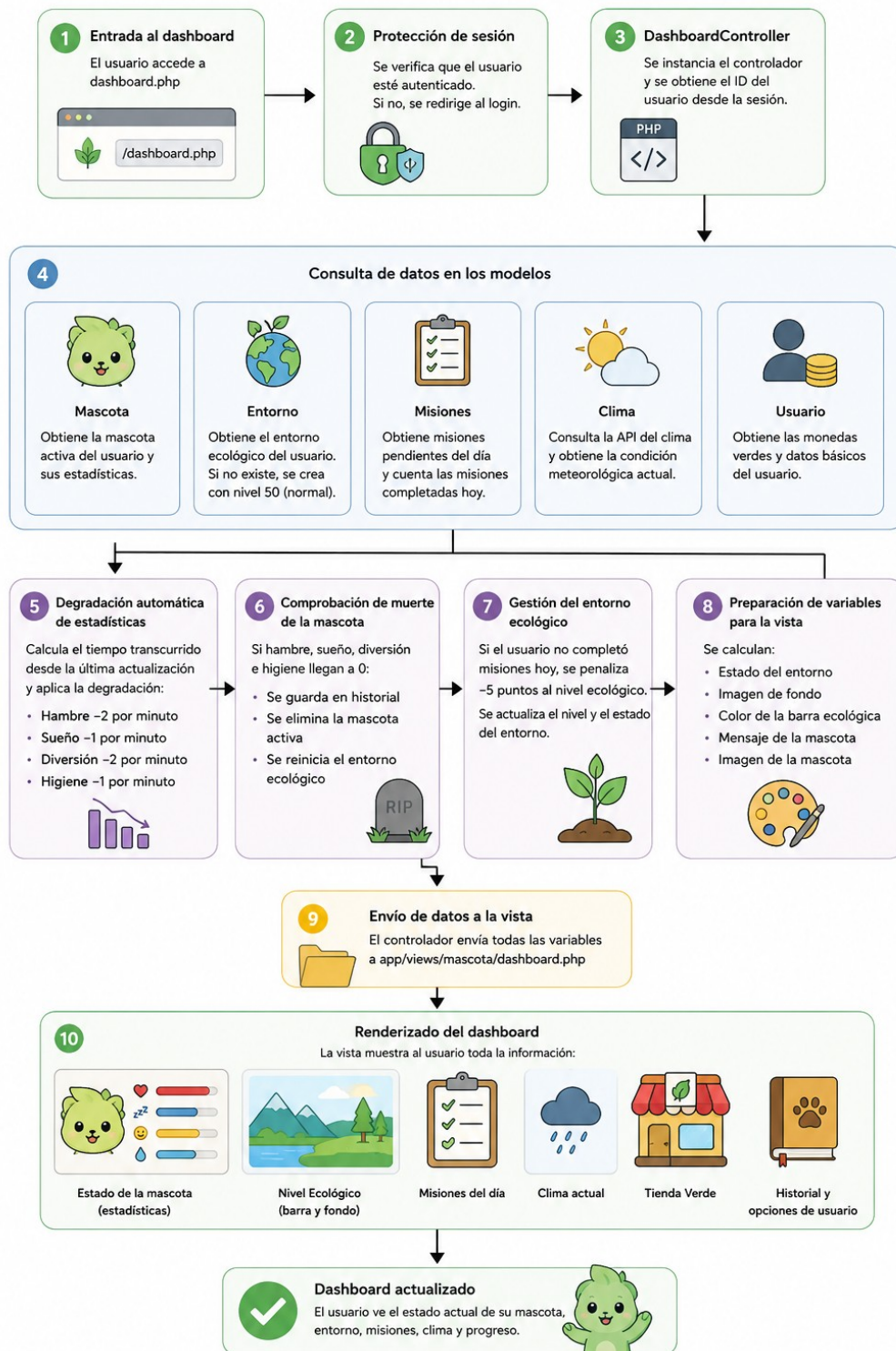


Ilustración 28. Flujo Funcional el Dashboard

3.8. Decisiones de diseño y organización

Durante el desarrollo de EcoGotchi se tomaron distintas decisiones técnicas y organizativas con el objetivo de mantener una aplicación funcional, visualmente atractiva y sencilla de ampliar. Muchas de estas decisiones estuvieron relacionadas con la evolución progresiva del proyecto, la organización interna del código y la experiencia de usuario dentro del sistema de gamificación ecológica.

3.8.1. Decisión de utilizar arquitectura MVC

Una de las decisiones técnicas más importantes del proyecto fue reorganizar progresivamente la aplicación utilizando una arquitectura inspirada en el patrón MVC.

Esta decisión permitió separar la lógica de negocio, la interfaz visual y el acceso a datos, facilitando el mantenimiento del código y el trabajo colaborativo entre los miembros del equipo.

Además, esta organización permitió incorporar nuevas funcionalidades de forma más ordenada conforme el proyecto fue creciendo.

3.8.2. Decisiones de diseño visual

Desde las primeras fases del proyecto se buscó crear una interfaz visual inspirada en videojuegos tipo Tamagotchi, utilizando colores llamativos, animaciones y elementos gráficos relacionados con la naturaleza y la sostenibilidad.

El dashboard se diseñó como el centro principal de interacción del usuario, mostrando de forma visual las estadísticas de la mascota, el estado ecológico, las misiones activas y las diferentes acciones disponibles.

También se incorporaron elementos dinámicos como:

- Fondos ecológicos variables,
- Sistema climático,
- Barra de estadísticas animadas,
- Mensaje de la mascota,
- Diseño responsive adaptado a distintos tamaños de pantalla.

Estas decisiones ayudaron a mejorar la experiencia de usuario y reforzar la sensación de progresión dentro del juego.

3.8.3. Decisiones sobre la jugabilidad

El sistema de juego fue diseñado utilizando mecánicas sencillas de gamificación con el objetivo de incentivar la interacción constante del usuario con la plataforma.

Para ello se implementaron distintas reglas internas como:

- Degradación automática de estadísticas con el paso del tiempo,
- Sistema de monedas verdes,
- Penalización ecológica por inactividad,
- Recompensas por completar misiones,
- Evolución visual del entorno dependiendo del nivel ecológico alcanzado.

Por ejemplo, si el usuario no completa misiones ecológicas durante el día, el sistema reduce automáticamente el nivel ecológico del entorno. Además, las estadísticas de la mascota disminuyen progresivamente conforme transcurre el tiempo desde la última conexión del usuario.

Estas decisiones permitieron generar una mayor sensación de responsabilidad sobre el cuidado de la mascota y el mantenimiento del entorno ecológico virtual.

3.8.4. Decisiones organizativas del equipo

A nivel organizativo, el desarrollo del proyecto se realizó de forma colaborativa mediante Git y GitHub, permitiendo dividir funcionalidades y mantener un control de versiones del sistema.

La organización modular del proyecto facilitó que diferentes miembros del equipo pudieran trabajar sobre distintas partes de la aplicación, como el sistema de mascotas, las misiones ecológicas, la Tienda Verde o la reorganización MVC del proyecto.

Además, el uso de ramas y commits permitió integrar nuevas funcionalidades progresivamente y mantener una mayor estabilidad durante el desarrollo.

4. Implementación técnica y Lógica de Negocio

En este apartado se detalla el proceso de construcción de la aplicación web EcoGotchi. Se describe la evolución desde el entorno de desarrollo local hasta las adaptaciones necesarias para la nube, profundizando en el stack tecnológico, la lógica de gamificación, la refactorización hacia el patrón MVC y el sistema de base de datos dual implementado.

4.1. Stack tecnológico avanzado

Para el desarrollo de la plataforma se ha optado por una combinación de tecnologías tradicionales y soluciones *cloud* modernas, asegurando una experiencia interactiva en el cliente y un procesamiento robusto en el servidor.

- **Backend (Servidor):** Se ha utilizado **PHP 8.2** como lenguaje principal. Gestiona la lógica de negocio, las sesiones de usuario y el cálculo algorítmico de la degradación de las mascotas. Para la conexión a la base de datos se emplea **PDO (PHP Data Objects)**, lo que aporta una capa de abstracción fundamental para nuestro entorno dual. Las contraseñas se gestionan mediante encriptación nativa con el algoritmo **bcrypt**.
- **Frontend (Cliente):** La interfaz se ha construido con **HTML5 y CSS3**, implementando un diseño *responsive* y soporte nativo para *dark mode*. La interactividad se apoya íntegramente en **JavaScript (Vanilla)**, evitando dependencias externas pesadas como jQuery. JavaScript controla los minijuegos, la mecánica de *drag & drop* para la limpieza del entorno, las llamadas asíncronas (Fetch API) y el sistema de notificaciones no intrusivas (Toast).
- **Base de Datos Dual:** **MySQL/MariaDB** para el desarrollo local mediante XAMPP y **PostgreSQL gestionado (Supabase)** para el entorno de producción.

4.2. Arquitectura y estructura del proyecto (Patrón MVC)

A medida que EcoGotchi escalaba en funcionalidades, se hizo imperativo abandonar la estructura de archivos procedurales independientes para adoptar una arquitectura inspirada en el patrón **Modelo-Vista-Controlador (MVC)**. Esta separación de responsabilidades facilita el mantenimiento y el trabajo colaborativo.

ProyectoFinalFP/

```
|— api/
|   └─ index.php          ← Router principal (Vercel serverless)
|— app/
|   └─ controllers/      ← Lógica de negocio (ej. DashboardController.php)
|   └─ models/           ← Consultas SQL (ej. Entorno.php, Mascota.php)
|   └─ views/            ← Plantillas de interfaz (auth, mascota, tienda)
|— assets/               ← Estilos, JS, sprites y fondos optimizados
|— config/
|   └─ database.php      ← Conexión dual (Local/Supabase)
└─ sql/                 ← Esquemas de BD (MySQL y PostgreSQL)
```

Para solventar el problema de las rutas al mover la aplicación entre entornos, se implementó la constante dinámica `ASSETS_URL`. Esta constante resuelve automáticamente la ruta hacia `/ProyectoFinalFP/assets` en local y hacia `/assets` en producción.



Ilustración 29. Diagrama de arquitectura del proyecto

4.3. Arquitectura Funcional y Módulos del Sistema

La plataforma EcoGotchi se articula mediante una serie de bloques lógicos interconectados que garantizan la eficiencia operativa y una experiencia de usuario optimizada:

- **Gestión de Identidad y Sesiones:** Comprende el flujo de registro y el sistema de autenticación mediante *hashes* de seguridad, regulando el acceso a las capas protegidas del servidor.
- **Ciclo de Vida de la Mascota:** Orquestación de la creación, el mantenimiento activo y la transición hacia el historial en caso de fallecimiento, habilitando el reinicio de la mecánica de juego.
- **Dashboard de Control Dinámico:** Interfaz centralizada que procesa en tiempo real las métricas vitales, la asignación de retos diarios y la ambientación climática.
- **Mecánicas de Sostenibilidad:** Algoritmos de gamificación donde la negligencia impacta negativamente en el ecosistema, mientras que el cumplimiento de misiones genera "Monedas Verdes".
- **Tienda de Impacto Real:** Entorno transaccional para la conversión de recompensas virtuales en incentivos alineados con la preservación del medio ambiente.
- **Gestión de Privacidad (Baja):** Funcionalidad de supresión de cuenta que asegura el borrado integral de registros conforme a los estándares legales de privacidad.

4.4. Base de Datos y Adaptación a Supabase

Uno de los mayores retos técnicos del proyecto fue la migración de un entorno puramente local a un entorno *serverless* en la nube. Mientras que en local (XAMPP)

utilizaba MySQL, para producción se optó por **Supabase**, un servicio de base de datos PostgreSQL gestionado, debido a su fiabilidad y perfecta integración con Vercel.

4.4.1. Conexión Dual y Transaction Pooler

El archivo config/database.php fue reescrito para implementar un patrón de conexión dual. El sistema detecta automáticamente si se encuentra en local (127.0.0.1) o en producción a través de las variables de entorno.

Dado que las funciones *serverless* de Vercel pueden abrir miles de conexiones simultáneas y agotar los recursos de la base de datos, se configuró Supabase para utilizar el puerto 6543, activando el **Transaction Pooler (PgBouncer)**. Esto asegura la compatibilidad IPv4 con Vercel y mantiene un grupo de conexiones reciclables. Para que PDO funcionara correctamente con el pooler en modo transacción, fue obligatorio activar la directiva PDO::ATTR_EMULATE_PREPARES => true.

4.4.2. Compatibilidad de Consultas SQL (MySQL vs PostgreSQL)

Al utilizar dos motores de bases de datos distintos, la capa de modelos (/app/models/) tuvo que diseñarse para ejecutar consultas agnósticas o condicionales. Se gestionaron las siguientes discrepancias sintácticas principales:

Función Lógica	Sintaxis MySQL (XAMPP)	Sintaxis PostgreSQL (Supabase)
Obtener fecha actual del sistema	CURDATE()	CURRENT_DATE
Extracción de fecha de un timestamp	DATE(campo_fecha)	campo_fecha::date

Función Lógica	Sintaxis MySQL (XAMPP)	Sintaxis PostgreSQL (Supabase)
Generación de valores aleatorios	RAND()	RANDOM()

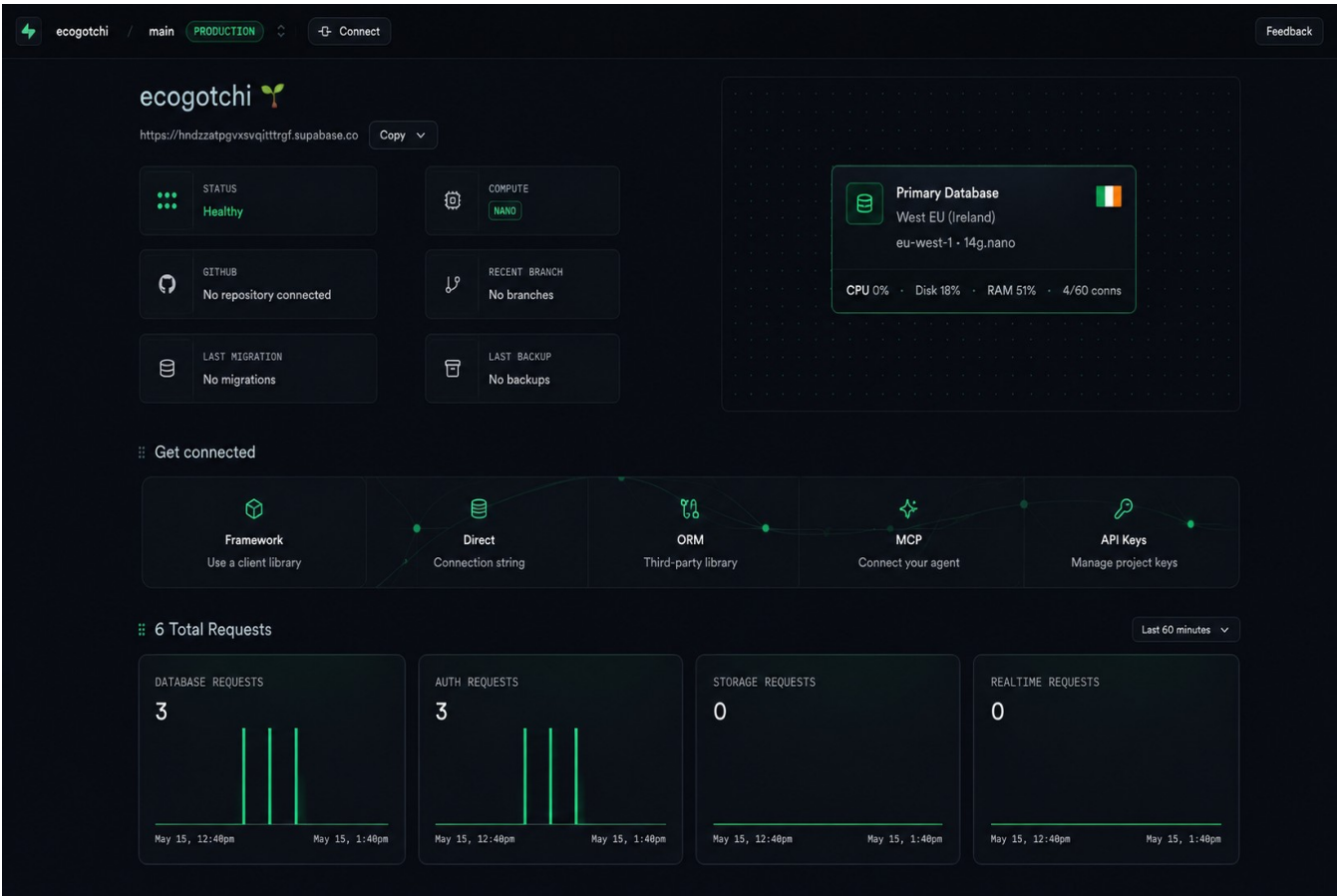


Ilustración 30. Dashboard Supabase

4.5. Lógica de Gamificación y Mecánicas Cliente-Servidor

El "motor" del juego de EcoGotchi se basa en la sincronización entre las acciones del cliente (JS) y la validación en el servidor (PHP).

4.5.1. Degradación y Muerte de la Mascota

Las estadísticas de la mascota (Hambre, Sueño, Diversión e Higiene) se degradan automáticamente. El cálculo no se basa en un *cron job*, sino en una evaluación temporal: al acceder al DashboardController, el sistema calcula la diferencia en minutos entre la fecha_ultima_actualizacion en la BD y el momento actual. Se restan proporcionalmente los puntos (ej. Hambre baja 2pts/min, Sueño 1pt/min). Si todas las estadísticas llegan a cero simultáneamente, el sistema ejecuta la rutina de muerte: la mascota pasa a la tabla mascotas_historial, se elimina de la tabla activa y se reinicia el entorno ecológico a nivel 50.

4.5.2. Minijuegos y Entorno Interactivo

Para aumentar el *engagement*, las acciones no son simples botones estáticos. Se desarrollaron mecánicas interactivas en JavaScript puro:

- **Limpieza de Entorno (Drag & Drop):** De forma aleatoria, aparece "basura" en la pantalla del entorno. Utilizando los eventos nativos de HTML5 dragstart, dragover y drop, el usuario debe arrastrar los residuos a un contenedor virtual. Al completarlo, JS hace una petición asíncrona a PHP para subir el nivel de higiene y el nivel ecológico.
- **Minijuegos de Misiones:** Para validar las misiones diarias (que se generan aleatoriamente para cada usuario sin repetirse), se crearon pequeños retos lógicos en JS: *Clic rápido* (medición de reflejos), *Secuencia de teclas* (memoria muscular) y *Acertijos de texto*.
- **Notificaciones Toast:** Se sustituyó el obsoleto alert() del navegador por un sistema de notificaciones *Toast* personalizadas en CSS y JS, mejorando enormemente la Experiencia de Usuario (UX) al informar sobre recompensas obtenidas sin interrumpir el flujo de juego.

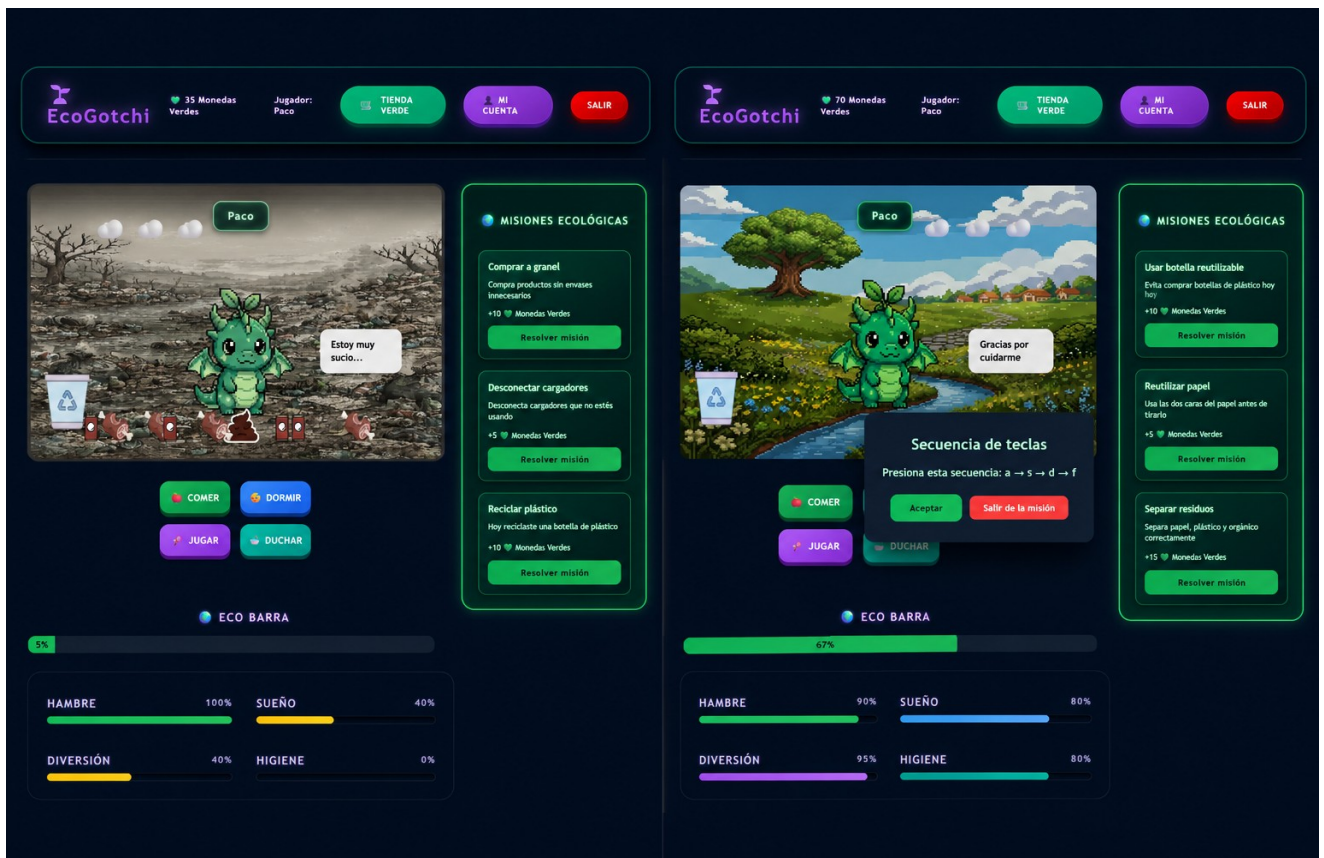


Ilustración 31. Limpieza de Entorno y Minijuegos

4.6. Control de Versiones y Gestión de Repositorios

La arquitectura de EcoGotchi ha sido centralizada mediante un repositorio alojado en la organización "MAFIA-DE-DAW". Esta elección estratégica ha posibilitado una administración profesional del código fuente, fomentando una dinámica de colaboración síncrona y eficiente entre los integrantes del proyecto.

Metodología de Trabajo y Sinergia Operativa:

Se ha establecido un ecosistema basado en Git que optimiza diversos ejes del desarrollo:

- **Trazabilidad Atómica de Cambios:** Aplicación de *commits* con una carga semántica profunda para monitorizar la evolución de componentes críticos, tales como el **DashboardController** o el módulo transaccional de la tienda.
- **Colaboración Multidisciplinar:** Preservación de la integridad estructural mediante ramas de desarrollo y procesos de *merge*, permitiendo que la lógica *frontend* y *backend* progresen de forma paralela y sin conflictos.
- **Estandarización de Documentación:** El archivo **README.md** actúa como el manual de referencia fundamental para el despliegue del entorno, operando como la interfaz técnica principal de la plataforma.

Despliegue Continuo (CI/CD) y Blindaje de Infraestructura:

GitHub se constituye como la base de nuestra arquitectura en la nube:

- **Automatización en Producción:** La vinculación directa con Vercel garantiza que cualquier actualización en la rama *main* desencadene un flujo de construcción inmediato.
- **Seguridad del Código:** Se aplica una política rigurosa para la protección de activos sensibles. Las credenciales de acceso a Supabase se han omitido del repositorio, delegando su gestión a las *environment variables* del servidor de producción.

5. Despliegue en Producción y Entorno Cloud

El paso final en el ciclo de vida del desarrollo de software es su puesta en producción. EcoGotchi no se limitó a una demostración local, sino que fue desplegado en una arquitectura *serverless* real a través de Vercel, conectada a la base de datos de Supabase.

5.1. Vercel Serverless Runtime y Enrutamiento (Router API)

Vercel es una plataforma optimizada para *frameworks* de frontend (como Next.js o React). Para ejecutar PHP de forma nativa en su entorno *serverless*, fue necesario configurar el runtime `vercel-php@0.7.2` mediante el archivo de configuración `vercel.json`. Sin embargo, esto plantea un desafío: Vercel no interpreta los archivos PHP fuera de la carpeta `/api` por defecto.

Para solucionarlo, se desarrolló un **Router Principal** (`api/index.php`). Este archivo actúa como un controlador frontal (Front Controller). Su función es triple:

- **Interceptar peticiones:** Recibe todas las rutas solicitadas por el cliente.
- **Servir estáticos:** Si la petición es hacia la carpeta `/assets` (CSS, JS, imágenes), el router lee el archivo con la función `readfile()` de PHP, inyectando previamente el Content-Type correcto en las cabeceras HTTP.
- **Enrutamiento limpio:** Si es una petición de vista o lógica, el router procesa URLs amigables (ej. `/dashboard` en lugar de `/dashboard.php`) y hace el *include* del controlador o archivo correspondiente dentro de la carpeta `/app/`.

5.2. Optimización de Assets para Serverless

Las arquitecturas *serverless* tienen límites estrictos en el tamaño de las funciones y el tiempo de ejecución. Inicialmente, los fondos del entorno ecológico de la aplicación eran imágenes PNG en alta resolución que ocupaban alrededor de 8MB cada una. Al procesarse mediante el router PHP, esto provocaba tiempos de carga excesivos (latencia) y riesgo de *Timeout* en Vercel.

Se implementó una política de optimización drástica: todas las imágenes de fondo fueron redimensionadas y convertidas a formato **JPEG optimizado**, reduciendo el peso a menos de 0.5MB por imagen. Los sprites de las mascotas y la tienda se mantuvieron con transparencias, pero comprimidos. Esto garantizó una carga casi instantánea del Dashboard.

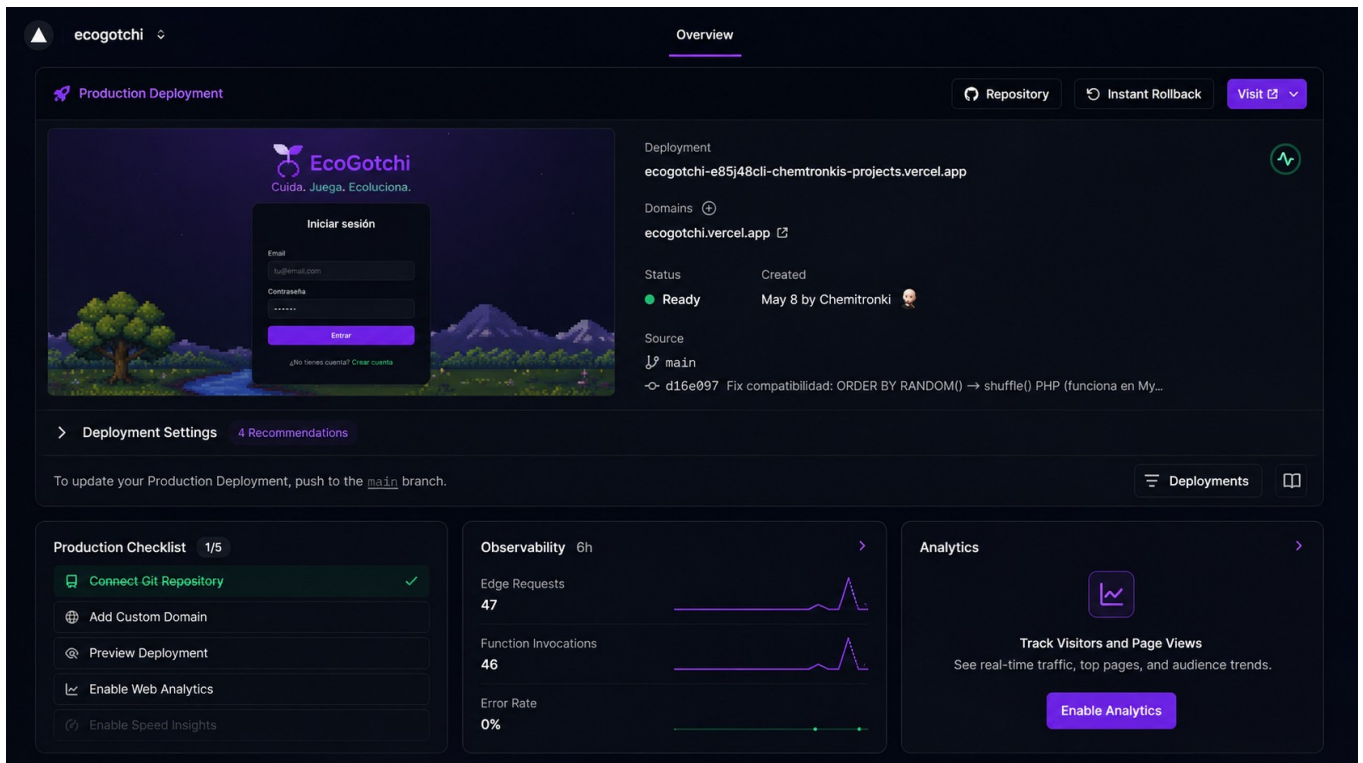


Ilustración 32. Proceso de despliegue en Vercel

5.3. Integración Continua (CI/CD) y Variables de Entorno

El despliegue se automatizó conectando el repositorio de **GitHub** directamente con Vercel. Este flujo de Integración Continua y Despliegue Continuo (CI/CD) asegura que cada *push* a la rama principal (main) desencadene un nuevo proceso de *build* en los servidores de Vercel, publicando la nueva versión en minutos sin intervención manual. Por motivos de ciberseguridad extrema, ninguna credencial de Supabase se subió a GitHub. En su lugar, se configuraron en la pestaña *Environment Variables* de Vercel.

El sistema lee en tiempo de ejecución las variables DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD, inyectándolas de forma segura en la conexión PDO.

5.4. Pruebas funcionales y de usuario.

Atendiendo a la naturaleza operativa y la envergadura del desarrollo, se ha prescindido de una infraestructura de pruebas unitarias automatizadas basadas en **PHPUnit**. La robustez del sistema se ha garantizado, por consiguiente, mediante la ejecución de rigurosos ciclos de validación funcional manual que auditan la integridad de cada módulo:

- **Autenticación y Seguridad:** Auditoría de los flujos de acceso para asegurar la integridad de las sesiones bajo diversos escenarios de entrada.
- **Persistencia del Ciclo de Vida:** Corroboración de la correcta escritura de metadatos y atributos en el esquema relacional durante el proceso de creación.
- **Algoritmo de Deterioro Temporal:** Monitorización del cálculo dinámico de métricas (ej. Hambre a 2pts/min) para certificar su precisión cronológica.
- **Ruptura y Transición de Estado:** Supervisión del procedimiento de migración hacia la tabla de registros históricos una vez alcanzado el umbral crítico de supervivencia.
- **Ejercicio del Derecho de Supresión:** Auditoría del borrado integral y transaccional de la identidad digital tras la petición de baja voluntaria.
- **Adaptabilidad de la Interfaz:** Análisis del rendimiento visual y la experiencia de usuario en ecosistemas de visualización multiplataforma.

6. Normativa, Ética y Licencias

6.1. Marco Legal: Privacidad y RGPD

Al procesar datos de registro, la aplicación cumple estrictamente con el Reglamento General de Protección de Datos (RGPD). Se han aplicado los siguientes principios técnicos de "Privacidad desde el Diseño":

- **Confidencialidad:** Como se mencionó, las contraseñas se almacenan empleando algoritmos de *hash* irreversibles (bcrypt). Una brecha de seguridad en la base de datos no comprometería las credenciales reales de los usuarios.
- **Derecho al Olvido:** En la sección `mi_cuenta.php`, el usuario dispone de un botón directo para ejercer su derecho de supresión. Al ejecutar esta acción, una consulta SQL con borrado en cascada (o eliminación transaccional) elimina permanentemente su cuenta, su historial de mascotas, sus misiones y su entorno, sin dejar registros residuales.

6.2. Impacto Ético: Gamificación y ODS

El diseño de EcoGotchi no busca la retención del usuario mediante tácticas psicológicas perjudiciales (Dark Patterns), sino a través de la **Gamificación Positiva**. El núcleo del sistema gira en torno a la "Tienda de Impacto Real". En lugar de comprar "sombreros" virtuales para la mascota, el usuario invierte sus "Monedas Verdes" (conseguidas mediante hábitos sostenibles) en acciones del mundo real representadas virtualmente, como la financiación de santuarios de abejas o la construcción de pozos solares.

Este enfoque alinea tecnológicamente el proyecto con la **Agenda 2030**, contribuyendo a la concienciación sobre el **ODS 12 (Producción y Consumo Responsables)** y el **ODS 13 (Acción por el Clima)**.

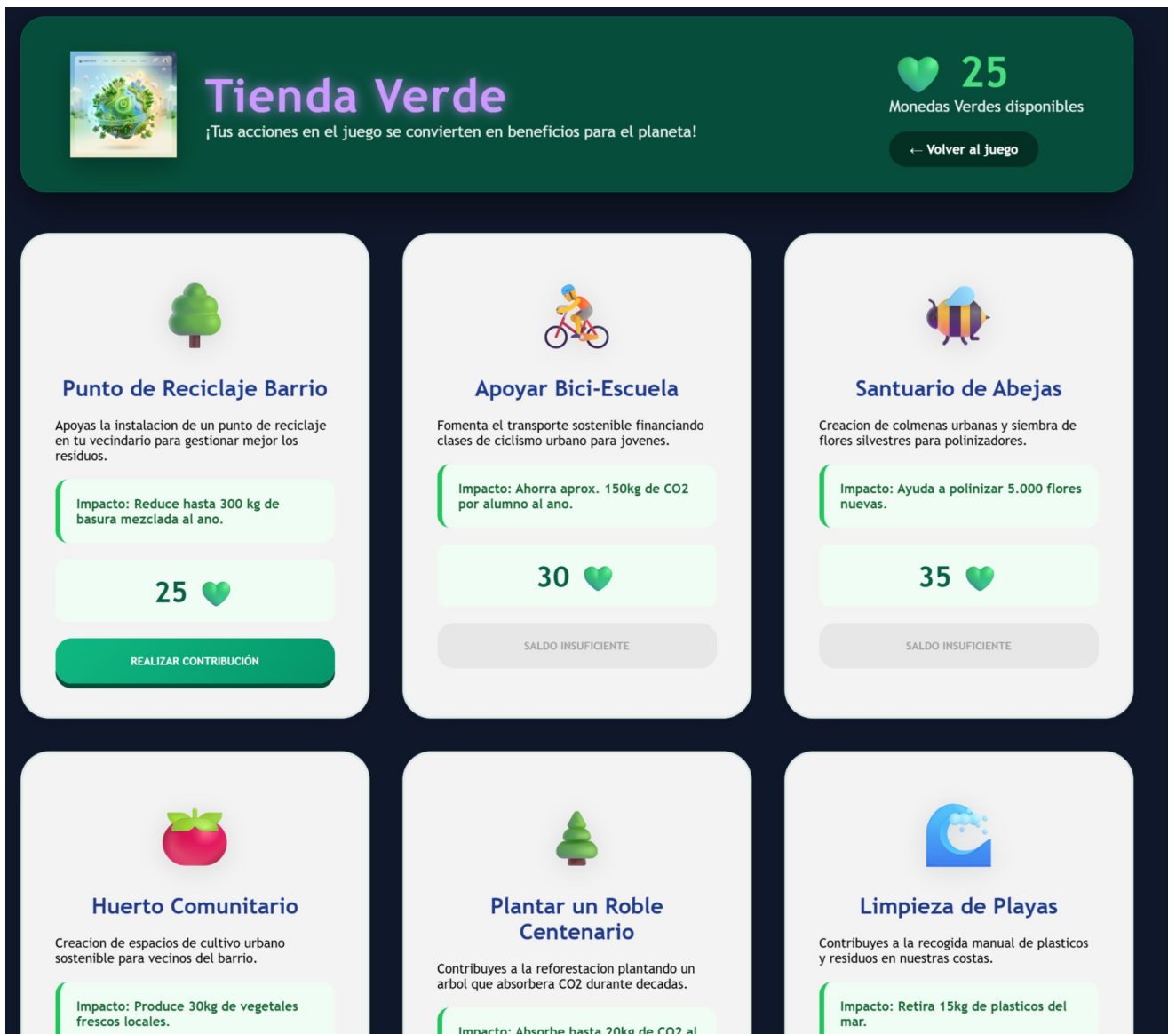


Ilustración 33. Tienda Eco Impacto Real

6.3. Propiedad Intelectual

El código de EcoGotchi, alojado en el repositorio de la organización "MAFIA-DE-DAW", se considera un recurso académico abierto. Se ha procurado el uso de librerías, tipografías e imágenes (fondos) libres de derechos (Dominio Público o licencias Creative Commons CC0) para evitar infracciones de *copyright* en su despliegue público

7. Conclusiones y Trabajo Futuro

7.1. Conclusiones y Cumplimiento de Objetivos

El desarrollo de EcoGotchi ha demostrado ser un éxito integral, superando las expectativas habituales de un Proyecto de Grado Superior. Se ha construido una plataforma completa (Full-Stack) funcional, estable e interactiva. El objetivo principal de utilizar la nostalgia de los *Tamagotchi* como vehículo para la educación ambiental se ha materializado a través del sistema de misiones y la Tienda de Impacto Real.

Técnicamente, se ha logrado implementar un sistema de autenticación seguro, una base de datos relacional robusta (tanto en MySQL como en PostgreSQL) y una arquitectura del lado del cliente altamente interactiva basada íntegramente en JavaScript Vanilla, sin sobrecargar la aplicación con dependencias innecesarias.

7.2. Lecciones Aprendidas

El mayor desafío formativo ha residido en la transición entre el entorno local y el despliegue en la nube. Las lecciones más valiosas incluyen:

- **Adaptabilidad de Bases de Datos:** Comprender y resolver las diferencias sintácticas entre motores de bases de datos (MySQL vs PostgreSQL) mediante consultas agnósticas ha sido un gran ejercicio de ingeniería de software.
- **Paradigma Serverless:** Aprender a configurar un enrutador PHP para funcionar en Vercel, lidiar con los límites de tamaño de las funciones y entender el funcionamiento de un Transaction Pooler (PgBouncer) para evitar la saturación de conexiones.
- **Refactorización Continua:** La transición hacia un modelo inspirado en MVC demostró en la práctica por qué la separación del código "espagueti" es fundamental para la escalabilidad del proyecto.

Además, la ejecución del proyecto EcoGotchi ha constituido un marco de aprendizaje excepcional para consolidar competencias transversales propias del Grado Superior en DAW:

- **Abstracción y Arquitectura de Software:** Implementación rigurosa del patrón **MVC** para garantizar el desacoplamiento efectivo entre la persistencia de datos y la capa de presentación.
- **Sinergia y Control de Versiones:** Administración profesional del ciclo de vida del código a través de Git, optimizando la resolución de conflictos y la trazabilidad mediante ramas estratégicas en GitHub.
- **Modelado Relacional Avanzado:** Diseño, normalización e integridad de esquemas **SQL** complejos para soportar la lógica transaccional del sistema.
- **Operaciones en la Nube (DevOps):** Dominio de paradigmas *serverless*, orquestación de variables de entorno y automatización de flujos **CI/CD** para entornos de producción reales.

7.3. Hoja de ruta: Evolución y Escalabilidad

Gracias a la arquitectura desacoplada del sistema, se plantean diversas estrategias de expansión para consolidar a EcoGotchi como una solución tecnológica global orientada a la sostenibilidad:

7.3.1. Transición hacia Ecosistemas Móviles (PWA y Nativo)

Con el objetivo de optimizar la retención y permitir una interacción fluida en tiempo real, se proyectan dos hitos de desarrollo móvil:

-Fase 1: Implementación de PWA. Integración de un *Service Worker* y configuración de manifest.json para habilitar la instalación directa en dispositivos iOS y Android. Esto facilitará el almacenamiento en caché para uso *offline* y el despliegue de notificaciones *push* ante umbrales críticos en las estadísticas de la mascota.

-Fase 2: Desarrollo Nativo/Híbrido. Migración hacia **Flutter o React Native** consumiendo el backend mediante una arquitectura de API. Esta evolución permitirá explotar el hardware del terminal, integrando el acelerómetro en mecánicas de juego y el **GPS** para recompensar el transporte sostenible.

7.3.2. Convergencia con IoT (Internet de las Cosas)

Se contempla vincular el bienestar de la mascota con métricas de consumo doméstico real. Mediante la conexión con **Smart Plugs** o teledetector eléctrica, el estado ecológico del avatar reflejará directamente el ahorro energético verificado en el hogar del usuario.

7.3.3. Dinámicas Sociales y Gamificación Grupal

Para potenciar la tracción social y el refuerzo positivo entre pares, se proponen las siguientes funcionalidades:

- **Escalafón Verde Global:** Un sistema de *ranking* público que destaque a los usuarios con mayor impacto ambiental positivo.
- **Comunidades de Impacto:** Desarrollo de clanes para abordar desafíos colectivos, donde el esfuerzo conjunto financie proyectos de restauración ecológica tangibles.
- **Interacción entre Usuarios:** Capacidad de visitar otros entornos virtuales para colaborar en tareas de mantenimiento, reforzando la dimensión cooperativa del proyecto.

7.3.4. Inteligencia Artificial y Algoritmos Adaptativos

Integración de modelos de **Machine Learning** para el análisis de patrones de usuario. El motor de juego podrá sugerir misiones con mayor tasa de conversión individual y ajustar la dificultad de los minijuegos según el progreso detectado por la IA.

7.3.5. Aseguramiento de la Calidad (QA) y Testing

Con el fin de alcanzar estándares de producción industrial, se planifica la implementación de un ciclo de pruebas robusto:

- **Pruebas Unitarias:** Empleo de **PHPUnit** para blindar la lógica de negocio contenida en modelos y controladores.
- **Pruebas de Integración (E2E):** Automatización de flujos críticos de usuario (desde el registro hasta la pasarela de la tienda) mediante herramientas como **Cypress**.

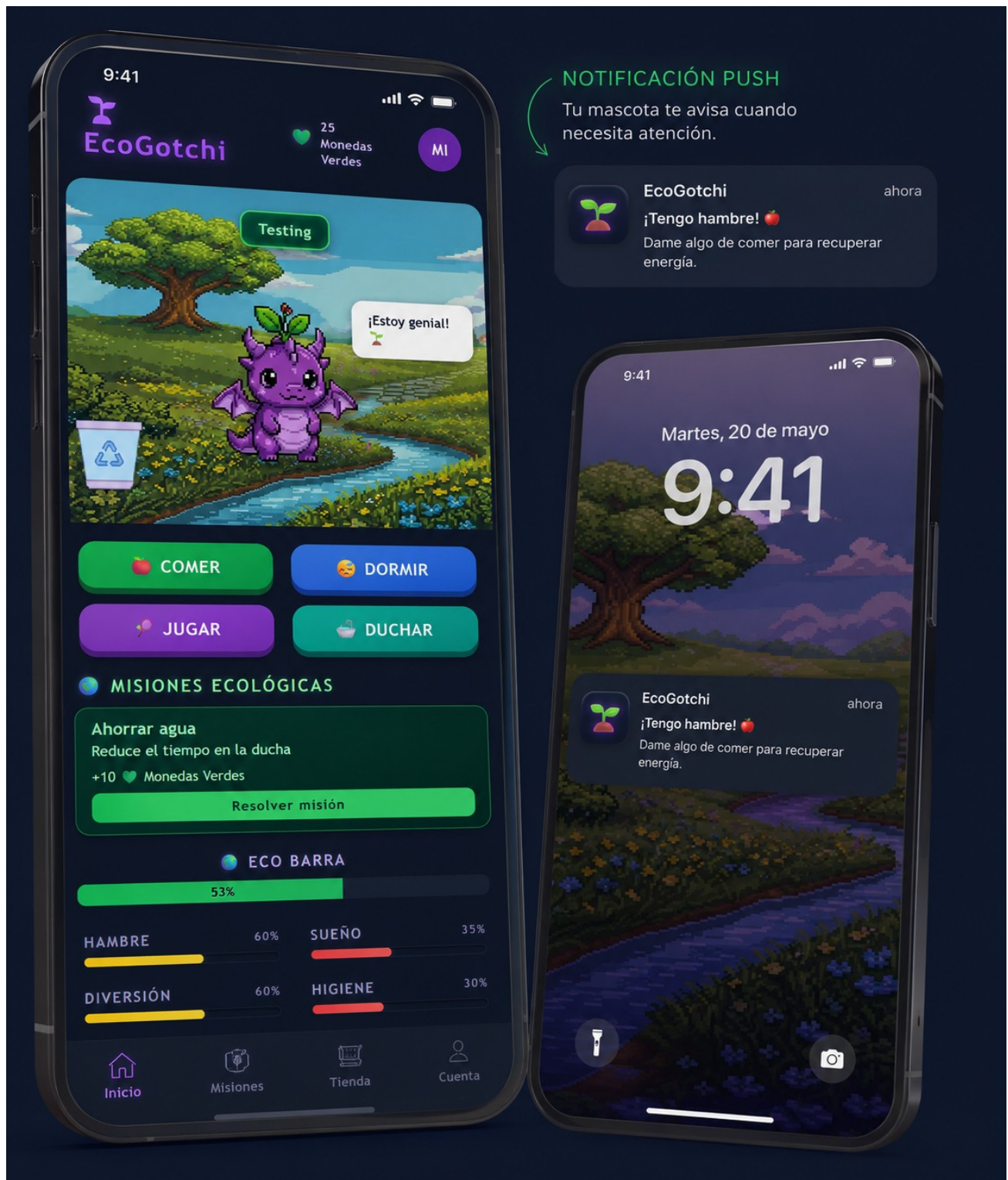


Ilustración 34. Mockup versión móvil

8. Bibliografía y Referencias Consultadas

1. Marco Normativo y de Privacidad

- Jefatura del Estado. (2018). **Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales**. *Boletín Oficial del Estado*, 294, de 6 de diciembre de 2018, pp. 119788-119857. <https://www.boe.es/eli/es/lo/2018/12/05/3>
- Parlamento Europeo y Consejo de la Unión Europea. (2016). **Reglamento (UE) 2016/679 (RGPD)**, relativo al tratamiento de datos personales y la libre circulación de los mismos. *Diario Oficial de la Unión Europea*, L 119, de 4 de mayo de 2016, pp. 1-88.

2. Sostenibilidad y Mecánicas de Juego

- Deterding, S., Dixon, D., Khaled, R., y Nacke, L. (2011). From game design elements to gamefulness: defining "gamification". *15th International Academic MindTrek Conference*, 9-15. <https://doi.org/10.1145/2181037.2181040>
- Naciones Unidas. (2015). **Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible**. Resolución de la Asamblea General (A/RES/70/1). <https://undocs.org/es/A/RES/70/1>
- Zichermann, G., y Cunningham, C. (2011). *Gamification by design: Implementing game mechanics in web and mobile apps*. O'Reilly Media.

3. Arquitectura Técnica y Desarrollo de Software

- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
- Mozilla Foundation. (2026). **MDN Web Docs**: Recursos técnicos sobre HTML5, CSS3 y JavaScript. <https://developer.mozilla.org/>
- Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.
- The PHP Group. (2026). *PHP Manual*: Documentación oficial del lenguaje. <https://www.php.net/manual/es/>

4. Gestión de Repositorios y Despliegue Cloud

- GitHub, Inc. (2026). *GitHub Docs*: Control de versiones y CI/CD. <https://docs.github.com/es>
- Vercel Inc. (2026). *Vercel Documentation*: Serverless Functions and Cloud Deployment. <https://vercel.com/docs>

5. Convenciones de Citación Académica

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.^a ed.). <https://doi.org/10.1037/0000165-000>